



Fakultät Informatik

KI-gestützte Transformation statischer in interaktive 3D-Objekte durch Benutzerinteraktion

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Friedrich Allenhof

Matrikelnummer 359 4928

Erstgutachter: Prof. Dr. Uwe Wienkop

Zweitgutachter: Louis Burk

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Inhaltsverzeichnis

1. Einführung	1
1.1. Problemstellung	1
1.2. Forschungsfragen und Zielsetzung	2
1.3. Abgrenzung	3
1.4. Aufbau der Arbeit	3
2. Grundlagen	5
2.1. Statische und interaktive 3D-Objekte	5
2.2. Transformation statischer in interaktive 3D-Objekte	6
2.3. Das Vivian Framework	6
2.3.1. Spezifikationen	7
2.4. Large Language Models und Agentische Systeme	7
2.4.1. Definition	7
2.4.2. Multimodale LLMs	8
2.4.3. Verwendung in der vorliegenden Arbeit	8
3. Stand der Forschung	11
3.1. Artikulierung von statischen Objekten	12
3.2. Bildbasierte Eingaben	12
3.3. Forschungslücke	13
4. Systemkonzeption und Architekturentscheidungen	15
4.1. Anforderungsanalyse	15
4.1.1. Funktionale Anforderungen	15
4.1.2. Nicht-funktionale Anforderungen	16
4.1.3. Domänenspezifische Anforderungen	16
4.2. Untersuchte Architekturansätze	17
4.2.1. Manager-gesteuerter Multi-Agenten-Ansatz	17
4.2.2. Begründung des modularen Workflow-Ansatzes	18
5. Prototypische Implementierung der gewählten Systemarchitektur	19
5.1. Systemkontext und Schnittstellen	19
5.2. Verwendete Sprachmodelle und Agenten	20
5.3. Gesamtpipeline und Datenfluss	22

5.4. Vorverarbeitung in Unity	23
5.5. Scene Analyzer	25
5.6. Interaction Planner	26
5.7. Specification Generator	29
5.8. Consistency Reviewer	35
5.9. Unity-Validator	38
6. Evaluation	41
7. Ausblick und Fazit	43
A. Agenten-Instruktionen	45
A.1. Scene Analysis Agent	45
A.2. Interaction Planning Agent	51
A.3. Consistency Review Agent	57
Abbildungsverzeichnis	61
Tabellenverzeichnis	63
Listingverzeichnis	65
Literatur	67

Kapitel 1.

Einführung

1.1. Problemstellung

In der Regel enthalten statische 3D-Objekte Geometrie, Materialien und Transformationen, jedoch kein Wissen darüber, welche expliziten Teile bedienbar sind, welche Funktion sie erfüllen oder wie sie auf Nutzer*innenhandlungen reagieren. Für Interaktivität sind aber genau diese Informationen von fundamentaler Bedeutung, nämlich wenn Objekte oder Teilobjekte mit Bedeutung, möglichen Handlungen und Verhalten verbunden werden [1]. Somit bestehen virtuelle, interaktive Prototypen nicht nur aus einer 3D-Geometrie, sondern besitzen zusätzlich semantisch-verhaltensbezogene Informationen.

Die Erstellung interaktiver 3D-Szenen oder die Transformation dieser in interaktive Prototypen setzt oft ein technisches Verständnis und Wissen voraus, wie etwa in 3D-Modellierung, Szenenaufbau oder Programmierung, was die Einstiegshürde für potenzielle Anwendende wie Produktentwickler*innen, Designer*innen oder Lehrende vergrößert. Diese Gruppe von Anwender*innen wird in dieser Arbeit als nicht-technische Benutzer*innen bezeichnet, also Personen ohne Kenntnisse in der Programmierung oder 3D-Modellierung. Die Fähigkeit fachliche Anforderungen und Funktionalität präzise formulieren zu können, wird dabei jedoch vorausgesetzt.

Interaktivität wird meist an konkrete Objekte oder Teilobjekte gebunden. Verfahren wie 3D-Scanning erlauben allerdings häufig nur eine monolithische Erzeugung statischer Abbildungen der Realität. Auch rekonstruierte Szenen aus Punktwolken bieten an sich keine einzeln adressierbaren Teile. Diese Abbilder sind somit überwiegend nicht semantisch segmentiert und Objekte existieren nicht als eigenständige, adressierbare Entitäten. Eine interaktive Anpassung oder Segmentierung im Nachhinein ist dann nur mit massivem manuellem Aufwand möglich.

Weiterhin muss Interaktionsverhalten in eine konsistente, maschinenlesbare Repräsentation überführt werden, da interaktive Prototypen mehr als eine rein textuelle Beschreibung des gewünschten Verhaltens benötigen. Diese Überführung ist in der Regel ebenfalls mit hohem manuellem Aufwand verbunden und dadurch auch fehleranfällig.

Außerdem fehlen in automatisierten Verfahren zur Erstellung und Bearbeitung von virtuellen Prototypen oft interne Prüf- oder Korrekturmaßnahmen, um Fehler frühzeitig und im Prozess erkennen und beheben zu können. Diese Fehler müssen dann häufig zeitaufwendig und manuell im Nachhinein behoben werden. Auch können Halluzinationen auftreten, wie etwa in 3D-LLMs, wodurch Objekte teils nicht existieren oder falsche Beziehungen zwischen Objekten bestehen [2].

Daraus lässt sich das zentrale Problem dieser Arbeit ableiten: Es fehlt ein niedrighschwelliger und dennoch kontrollierbarer Ansatz, mit dem bereits vorhandene statische 3D-Objekte in maschinenlesbare, validierbare und zustandsbasierte Prototypen überführt werden können, während die Geometrie beibehalten wird.

1.2. Forschungsfragen und Zielsetzung

Um den in 1.1 beschriebenen Problemen zu begegnen, untersucht die vorliegende Arbeit die Möglichkeit, einen Ansatz zu entwickeln, welcher die automatisierte Überführung statischer 3D-Objekte in validierte, Zustands-behaftete, interaktive Prototypen ermöglicht, der insbesondere auch nicht-technischen Benutzer*innengruppen eine Nutzung ermöglicht und die Geometrie der Teilobjekte beibehält. Im Zentrum steht die folgende Forschungsfrage:

Lässt sich eine automatisierte Pipeline konzipieren und prototypisch umsetzen, die vorhandene statische 3D-Objekte mittels natürlicher Sprache in interaktive Prototypen überführt und dabei vorhandene Geometrien beibehält?

Zur detaillierten Beantwortung dieser Frage werden die folgenden untergliederten Forschungsfragen definiert:

- **F1:** Lässt sich ein Co-Creation-Workflow konzipieren und prototypisch umsetzen, in dem natürlichsprachliche Beschreibung und Korrekturschleifen die Überführung statischer 3D-Objekte in interaktive Prototypen unterstützen?
- **F2:** Lassen sich relevante Objekte, Teilobjekte und semantische Rollen aus vorhandenen Szeneninformationen und Nutzer*innenbeschreibungen ableiten?
- **F3:** Können komplexe Interaktionslogiken aus der Kombination von bestehenden 3D-Geometrien und natürlicher Sprache extrahiert und in eine maschinenlesbare Repräsentation überführt werden?
- **F4:** In welchem Ausmaß reduzieren Validierungs- und Selbstkorrekturmaßnahmen syntaktische, semantische und referenzielle Fehler sowie fehlgeschlagene Unity-Validierungen im Vergleich zu einem Erstgenerierungsdurchlauf ohne Korrekturschleife?

Das Ziel dieser Arbeit ist die Konzeption und prototypische Implementierung einer KI-gestützten Pipeline, welche eine Überführung von statischen 3D-Objekten in interaktive Prototypen vornimmt und dabei natürliche Sprache entgegennimmt, um auch nicht-technischen Benutzer*innengruppen einen Zugang zu bieten. Dabei sollen die Fähigkeiten von LLMs genutzt werden und ein Co-Creation-Workflow entstehen, um einen Menschen in den Generierungsprozess einzubeziehen.

1.3. Abgrenzung

Diese Arbeit zielt nicht darauf ab, 3D-Szenendaten oder Geometrie zu generieren. Das beinhaltet komplette 3D-Szenen sowie einzelne 3D-Objekte, wie es in einigen verwandten Arbeiten der Fall ist. Es wird ausschließlich die Spezifikationsableitung bereits vorhandener virtueller Szenen betrachtet, um virtuelle Prototypen durch Verwendung des Vivian Framework zu erschaffen.

1.4. Aufbau der Arbeit

Aufbauend auf den in Kapitel 1 genannten Problemstellungen und Forschungsfragen werden in Kapitel 2 einige theoretische Grundlagen erläutert, wie statische und interaktive 3D-Objekte, das Vivian-Framework sowie LLMs und agentische Systeme, um ein fachliches Fundament für diese Arbeit zu bilden. In Kapitel 3 wird der derzeitige Stand der Forschung zur Verwendung von LLMs in virtuellen 3D-Welten eingeordnet. Darauf aufbauend wird die Forschungslücke herausgearbeitet, aus der sich die Relevanz der in Kapitel 4 entwickelten Pipeline ergibt. Anschließend beschäftigt sich Kapitel 5 mit der Implementierung und technischen Umsetzung dieser.

Kapitel 2.

Grundlagen

2.1. Statische und interaktive 3D-Objekte

Im Folgenden werden statische sowie interaktive 3D-Objekte definiert und erklärt. Die Begriffe 3D-Objekt und 3D-Asset werden in dieser Arbeit simultan verwendet. Beide beschreiben ein Element einer virtuellen Szene und können als alleinige Geometrie existieren oder Unterkomponenten bzw. Teilobjekte besitzen.

Statische 3D-Objekte

In dieser Arbeit werden statische 3D-Objekte als Basisform digitaler Modelle betrachtet. Sie sind die fundamentale Basis, auf denen komplexere Verhaltensweisen aufbauen können, jedoch wohnt ihnen mit diesen Informationen noch kein Eigenleben inne. Statische 3D-Objekte lassen sich meist rein geometrisch beschreiben. Die Mesh-Geometrie besteht aus Dreiecksnetzen oder Polyeder-Darstellungen, welche die Oberfläche durch Eckpunkte (Vertices) und deren Verbindungen definieren [3]. Weiterhin besitzen sie grafische Attribute wie Materialeigenschaften, Texturen, Transparenz und Sichtbarkeit sowie eine hierarchische Anordnung im Szenengraph, welcher ihre Position, Rotation und Skalierung organisiert [4].

Interaktive 3D-Objekte

Während statische 3D-Objekte als unbelebte geometrische Körper betrachtet werden können, zeichnen sich interaktive 3D-Objekte durch Wissen über ihre Funktionalität und die Möglichkeit zur Interaktion mit ihnen aus [1]. Auf diese 3D-Objekte kann durch Eingaben (Input) eingewirkt werden und das Objekt reagiert zustands- oder verhaltensbezogen [5]. Diese Reaktion verleiht der unbelebten Hülle ein „Leben“. Es ist somit in der Lage, auf durch Benutzereingaben ausgelöste Ereignisse (Events) zu reagieren und wird zu einem handlungs-offenen Teil der virtuellen Welt.

2.2. Transformation statischer in interaktive 3D-Objekte

Die in der vorliegenden Arbeit untersuchte Transformation von statisch zu interaktiv ist keine rein geometrische oder visuelle Modifikation, sondern eine semantisch-verhaltensbezogene. Sie macht aus einem visuellen und geometrischen Asset ein interaktives Modell, welches Informationen über sein eigenes Verhalten und die möglichen Interaktionen mit diesem beinhaltet. Mit dem Konzept dieser „Smart Objects“ wird Logik direkt im Objekt gespeichert, was wiederum Dezentralisierung und Wiederverwendbarkeit ermöglicht [1]. Dieser Prozess umfasst verschiedene Ebenen.

Zunächst wird das statische 3D-Modell segmentiert sowie semantisch etikettiert, was sich mit dem Begriff „Labeling“ zusammenfassen lässt. Dabei werden rohe Mesh-Geometrien in bedeutungsvolle Teile mit bestimmten Rollen zerlegt (z.B. der Griff einer Tasse oder die Sitzfläche eines Stuhls), sodass ein System später „versteht“, welches Teil eines 3D-Objekts für welche Interaktion relevant ist und eine Zuordnung stattfinden kann [6][7].

Im nächsten Schritt erfolgt der Übergang von der Semantik zum Verhalten mittels Zuweisung spezifischer Interaktionsmerkmale (Interaction Features) zu den im ersten Schritt analysierten semantischen Etiketten. Diese Merkmale beinhalten eine Beschreibung der Funktionalität sowie eine Verknüpfung mit Verhaltensplänen oder Zustandsautomaten. Diese Pläne können Zustände überprüfen oder ändern und legen das erwartete Verhalten von beteiligten Akteuren fest [1].

2.3. Das Vivian Framework

Das Vivian Framework ist ein Virtual-Prototyping-Framework. Es erstellt virtuelle Prototypen, also eine digitale, interaktive Repräsentation physischer Objekte aus der realen Welt. Diese Repräsentation kann z.B. in der Produktentwicklung verwendet werden, um virtuelle Prototypen von echten Produkten zu erzeugen und deren Verhalten frühzeitig zu testen, ohne dass das physische Produkt vorhanden sein muss. Diese virtuellen Prototypen funktionieren spezifikations-getrieben, sodass Verhalten nicht programmatisch, sondern deklarativ als Spezifikation geschrieben wird. Zur Laufzeit wird diese Spezifikation dann mittels einer State-Machine-basierten Umgebung interpretiert um ein Interaktionsverhalten abzubilden [8]. Ein Vorteil einer solchen spezifikations-getriebenen Vorgehensweise ist, dass beispielsweise Produktentwickler*innen keine Programmierkenntnisse benötigen, um virtuelle Prototypen von Produkten (Digital Twins) zu erstellen. So kann das Verhalten eines Produktes bereits frühzeitig und iterativ getestet werden. Ein weiteres mögliches Einsatzgebiet ist das Usability Engineering, mit dem interaktive Systeme so gestaltet und geprüft werden, dass sie für die jeweilige Zielgruppe möglichst effektiv, effizient sowie zufriedenstellend nutzbar sind. Dafür werden Systeme nicht erst am Ende, sondern bereits in der

Entwicklung, beispielsweise mittels Prototypen, auf Gebrauchstauglichkeit analysiert und getestet [9].

2.3.1. Spezifikationen

Wie bereits gezeigt, besitzen statische 3D-Objekte von sich aus keine Interaktivität (siehe 2.1). Das Vivian Framework bildet die Brücke zwischen statischem und interaktivem 3D-Objekt über eine maschinenlesbare Beschreibung des Verhaltens bei Interaktion mit dem virtuellen Prototyp. Die Verbindung dieser Spezifikation zu einem 3D-Objekt in der Szene wird über Namensreferenzen hergestellt. Ein Element in der Spezifikation referenziert demnach ein gleichnamiges 3D-Objekt oder Teilobjekt in der Szene. Weiterhin ermöglichen Spezifikationen eine Entkopplung des Verhaltensmodells vom 3D-Objekt, sodass unterschiedliche Spezifikationen verschiedene Verhalten bei demselben 3D-Objekt ermöglichen. Sie besteht im Wesentlichen aus 4 Teilen: Interaktionselemente (Interaction Elements), Visualisierungselemente (Visualization Elements), Zustände (States) und Übergänge (Transitions) [8].

Interaktionselemente. Hier befinden sich Elemente, welche Eingaben akzeptieren und somit Interaktion ermöglichen. Das können u. a. Buttons, Slider oder Movables sein.

Visualisierungselemente. Dabei handelt es sich Elemente, welche eine visuelle Rückmeldung geben. Typisch sind hier Light, Screen oder Sound Source.

Zustände. In diesem Teil befinden sich alle möglichen Zustände, die ein Prototyp erreichen kann. Jeder Zustand beschreibt eine bestimmte Situation, indem er Interaktions- und Visualisierungselementen Werte zuweist. Diese Zuweisung kann zudem an Bedingungen geknüpft sein.

Übergänge. Übergänge bestimmen, wie sich ein virtueller Prototyp von einem Zustand in einen anderen verändert. Diese Übergänge können durch Zeitüberschreitungen oder Ereignisse (Events) ausgelöst werden. Zusätzlich kann ein Übergang an eine zu erfüllende Bedingung (Guard) geknüpft sein.

2.4. Large Language Models und Agentische Systeme

2.4.1. Definition

Large Language Models (LLMs) sind tiefe neuronale Netzwerke, welche die Verarbeitung von natürlicher Sprache ermöglichen [10]. Sie enthalten typischerweise Milliarden von Parametern und sind in der Lage komplexe Aufgaben zu verstehen und zu lösen oder in Teilaufgaben zu zerlegen [11]. Ihre Skalierbarkeit ist ein zentrales Merkmal von LLMs, denn sie zeigen

mit zunehmender Modellgröße und Datenmenge qualitative Leistungssteigerungen und sogenannte emergente Fähigkeiten, wie z. B. *In-Context Learning*. Während solche LLMs als Modelle zur passiven Sprachverarbeitung und -generierung verstanden werden können, besitzen sie ohne zusätzliche Werkzeuge keine eigene Handlungsfähigkeit. Hier greift das Konzept eines LLM-basierten Agenten weiter. Ein Agent ist eine autonome Einheit, welche in einer Umgebung agiert, Entscheidungen trifft und auch mit anderen Agenten interagieren kann [12]. LLM-basierte Agenten verbinden diese klassische Idee mit den Fähigkeiten von LLMs. Ein LLM kann hier als kognitive Kernkomponente verstanden werden und verleiht dem Agent die Fähigkeit eigenständig und unabhängig zu analysieren, zu planen sowie Probleme zu lösen [13]. So entsteht ein System, welches nicht nur natürliche Sprache verarbeiten kann, sondern eine eigene zielgerichtete Handlungsfähigkeit besitzt und in einer Umgebung agieren kann. Ebenso kann es zusätzliche Komponenten wie Sensorik, API-Aufrufe oder Werkzeuge nutzen [13]. Die universelle Schnittstelle für solche agentischen Systeme, über die formuliert, geplant und koordiniert wird, ist die natürliche Sprache [14].

2.4.2. Multimodale LLMs

Multimodale Large Language Models (MLLMs) sind Modelle, welche im Gegensatz zu reinen LLMs nicht nur Text, sondern zudem Informationen wie Bild, Audio oder Video empfangen (Input), verarbeiten sowie ausgeben (Output) können. Eine für diese Arbeit wichtige Fähigkeit ist ihr Verstehen von 2D-Bildern und darüber zu schlussfolgern. Sie basieren auf LLMs und bestehen im Wesentlichen aus 3 Hauptmodulen: einem vortrainierten Encoder (Modality-Encoder), einem LLM sowie einem Adapter (Connector). Der Encoder verarbeitet die multimodalen Signale vor und der Adapter bildet die Brücke zum LLM, indem er diese Signale für das LLM verständlich umwandelt. Dieser Schritt ist nötig, da LLMs (wie in 2.4 gezeigt) ausschließlich Text verarbeiten können. Somit sind MLLMs in der Lage, Text sowie weitere multimodale Inhalte gemeinsam zu verstehen und darauf zu antworten [15], [16]. Im Vergleich zu früheren, ähnlichen Arbeiten basieren MLLMs auf LLMs mit Milliarden Parametern sowie neuen Trainingsparadigmen, wie dem „Multimodal Instruction Tuning“ [17]. Damit sind eine Vielzahl von Möglichkeiten gegeben, welche mit reinen LLMs nicht oder nur eingeschränkt realisierbar wären. Solche repräsentativen Fähigkeiten sind u. a. das Verständnis von visuell geprägten Witzen/Memes [18], [19], räumliches/koordinatenbezogenes Verständnis, das Erstellen von Website-Code basierend auf handgezeichneten Entwürfen oder Kochrezepte anhand von Bildern zubereiteter Gerichte [19].

2.4.3. Verwendung in der vorliegenden Arbeit

In dieser Arbeit werden LLM-basierte Agenten als Steuerungseinheiten in einer Pipeline zur Spezifikationsgenerierung für das Vivian Framework verwendet. Sie übernehmen eine

zentrale Rolle in der Übersetzung von natürlicher Sprache in strukturierte, maschinenlesbare Spezifikationen. Sie analysieren 3D-Szenen, planen mögliche Interaktionen mit einem 3D-Objekt, Erzeugen Spezifikationen zur Realisierung dieser Interaktionen und validieren semantische Korrektheit der erzeugten Artefakte.

Kapitel 3.

Stand der Forschung

Die aktuelle Forschung in diesem Gebiet verbindet Sprachmodelle mit 3D-Repräsentationen und zeigt, dass LLM-basierte Systeme zunehmend in der Lage sind, 3D-Szenen nicht nur mit Sprache zu beschreiben, sondern semantisch sowie räumlich zu interpretieren. Mit der Weiterentwicklung von LLMs konnte deren Integration mit räumlichen Daten große Fortschritte erzielen und bietet so neue Möglichkeiten wie Szenenverständnis, Planung und Generierung von 3D-Welten oder Navigation in diesen. Spätestens seit 2023 hat sich die Verbindung zwischen LLMs und 3D-Repräsentationen zu einer eigenständigen Forschungslinie entwickelt, welche 3D-Szenen nicht nur beschreiben, sondern auch für neue Methoden wie das Grounding, Fragebeantwortung und Planungsaufgaben nutzen können [20], [21]. Trotz dieser schnellen Entwicklung bleiben Datenknappheit, Halluzinationsrobustheit und räumliches Verständnis (Grounding) zentrale Herausforderungen aktueller Systeme [2], [20].

3D-LLMs für 3D-Verstehen. In Arbeiten wie 3D-LLM und PointLLM können multi-modale LLMs 3D-Daten direkt verarbeiten und nehmen dafür beispielsweise Punktwolken als Eingabe entgegen, wobei PointLLM eher objektzentriert ist. Dadurch können Aufgaben wie 3D-Captioning, 3D-Fragebeantwortung, Grounding, Dialog oder Navigation bearbeitet werden und relevante komplexere Konzepte wie räumliche Beziehungen zwischen Objekten, physikalische Eigenschaften, Raumlayouts oder Interaktionsmöglichkeiten (Affordances) verstehen [21], [22].

LLM-gestützte Pipelines zur Szenengenerierung und -bearbeitung. Parallel dazu wurden LLM-gestützte Pipelines entwickelt, in denen LLMs nicht direkt 3D-Daten verarbeiten. Stattdessen fungieren sie als Planungs- und Steuerungseinheit, die für die Generierung und Bearbeitung ganzer 3D-Szenen genutzt werden. Das Framework LLMR nutzt für die Generierung von interaktiven Szenen einen modularen Workflow, darunter die Module Planner, Scene Analyzer, Skill Library, Builder und Inspector. Es kann Szenenkontext analysieren und generiert anschließend Unity-kompatiblen C#-Code, welcher durch ein Inspektionsmodul iterativ geprüft wird. Als Eingabe arbeitet das System mit einer strukturierten, für das LLM konsumierbaren JSON-Repräsentation des Szenenkontexts [23]. HOLODECK verfolgt

einen stärker szenenzentrierten Ansatz und zerlegt die Generierung einer Szene in einzelne Schritte für Grundrisse, Materialien, Fenster und Türen, Objektauswahl sowie räumliche Anordnung. Es dient der automatisierten Erstellung von 3D-Simulationsumgebungen im Bereich der verkörperten KI (*Embodied-AI*) und platziert durch ein Constraint-basiertes Verfahren Objekte räumlich plausibel im Raum [24].

3.1. Artikulierung von statischen Objekten

Für die Transformation statischer in interaktive Objekte sind objekt- und szenenzentrierte Ansätze besonders relevant. ArtLLM wandelt eine Punktwolke in eine Abfolge von Tokens um, die vom Modell nacheinander generiert werden. Darüber hinaus können Assets auch als Bild oder Text eingegeben werden. Wichtig ist dabei, dass diese nicht direkt in das Kernmodell eingehen, sondern vorher mit Hilfe externer Generierungsmodelle in initiale Meshes und dann in Punktwolken umgewandelt werden. Diese Struktur definiert die Positionen der einzelnen Teile, die mechanischen Gelenke sowie deren Hierarchie. Um Positionen und Winkel anzugeben arbeitet es mit festen Rasterstufen statt Gleitkommazahlen. Nachdem detailliertere Geometrie über ein Zusatzmodul hinzugefügt wurde, wird diese kinematische Struktur als URDF-Datei (Unified Robotics Description Format) exportiert und kann so in Simulationen und virtuellen Umgebungen verwendet werden [25].

ArtiWorld erweitert diesen Ansatz von der Objekt- auf die Szenenebene. Das System nimmt eine textuelle Szenenbeschreibung entgegen und identifiziert auf dieser Grundlage artikulierbare Objekte. Anschließend erzeugt es strukturelle Beschreibungen der einzelnen Beziehungen und anschließend ebenfalls vollständige URDF-Dateien, welche an die ursprünglichen Koordinaten des starren 3D-Assets angepasst werden [26]. Die Geometrie des Originals wird dabei nicht ersetzt, sondern um Interaktivität erweitert. Insbesondere diese Arbeit zeigt, dass strukturelle Zwischenrepräsentationen das Reasoning verbessern.

3.2. Bildbasierte Eingaben

Eingaben basierend auf Bildern gewinnen in artikulationsbezogenen Pipelines an Bedeutung. Einzelbilder werden aktuell in mehreren Arbeiten genutzt, um simulierbare Szenen oder artikulierbare Objekte zu generieren, wie etwa in URDFormer oder ArtLLM [25], [27]. Dabei können Bilder sowohl isoliert, wie etwa in URDFormer, als auch zur ergänzenden Eingabe, wie in ArtLLM, verwendet werden, da diese semantische Hinweise und Informationen zu potenziellen Interaktionsmöglichkeiten bieten. Für die vorliegende Arbeit ist dieser Befund besonders relevant. Er zeigt die Kombination aus expliziten Raumdaten in Form von strukturellen Repräsentationen wie Punktwolken oder Szenengraphen mit Transformationen und Hierarchien einerseits sowie Bildern als visuellem Grounding andererseits.

3.3. Forschungslücke

Die in diesem Kapitel betrachteten Arbeiten zeigen, dass LLMs und MLLMs zunehmend in 3D-bezogenen Aufgabenbereichen eingesetzt werden und viele relevante Teilansätze wie 3D-Szenenverständnis, 3D-Fragebeantwortung, 3D-Grounding, Interaktions- und Aufgabenplanung, textbasierte 3D-Verarbeitung, Szenengenerierung sowie artikulationsbezogene Objektmodellierung existieren. So adressiert 3D-LLM u. a. 3D-Captioning, 3D-Fragebeantwortung, 3D-Grounding, Dialog und Navigation [21]. LLMR nutzt LLMs zur Erstellung und Modifikation interaktiver 3D-Welten zusammen mit Modulen wie Planner, Scene Analyzer, Builder und Inspector [23]. Holodeck erzeugt 3D-Umgebungen aus Sprache und nutzt ebenfalls LLMs u. a. für Objektwahl und räumliche Layout-Constraints [24]. ArtiWorld und URDFormer adressieren artikulationsbezogene bzw. URDF-basierte Simulation und Kinematik [26], [27]. Diese Ansätze adressieren jedoch meist andere Zielartefakte als die vorliegende Arbeit und erzeugen häufig neue 3D-Geometrien, bearbeitete 3D-Assets, vollständige 3D-Szenen, Unity-Code, URDF-Dateien oder robotische Simulationsumgebungen. Die vorliegende Arbeit ist im Gegensatz dazu nicht primär als Verfahren zur Geometrie- oder Szenengenerierung einzuordnen, sondern als semantisch-verhaltensbezogener Ansatz zur Ableitung von Interaktionsspezifikationen aus vorhandenen 3D-Szenen.

Die Forschungslücke ergibt sich damit aus Verbindung mehrerer Aspekte. Es sollen vorhandene statische 3D-Objekte erhalten bleiben, relevante Objekte und Teilobjekte semantisch interpretiert werden und gewünschtes Verhalten soll natürlichsprachlich beschrieben werden können. Darüber hinaus ergibt sich aus den Randbedingungen des Vivian Frameworks, dass eine maschinenlesbare Interaktionslogik entstehen soll, welche kompatibel mit dem Vivian-Framework ist. Da LLM-gestützte Generierungsprozesse fehlerhafte Objektverweise, inkonsistente Strukturen oder unzulässige Spezifikationselemente erzeugen können, sind zudem Validierungs- und Korrekturmaßnahmen erforderlich. Genau diese Kombination wird im derzeitigen Stand der Forschung nicht direkt adressiert. Die Problemstellung in Kapitel 1 nennt bereits, dass statische 3D-Objekte zwar Geometrie, Materialien und Transformationen besitzen, es aber an Wissen über bedienbare Elemente, Funktionen und Reaktionen auf Nutzer*innenhandlungen fehlt. Dort wird außerdem bereits ein niedrigschwelliger und kontrollierbarer Ansatz gefordert, welcher vorhandene statische 3D-Objekte in maschinenlesbare, validierbare und zustandsbasierte Prototypen überführt, während die Geometrie erhalten bleiben soll.

Aus dieser Forschungslücke sowie den Forschungsfragen F1 bis F4 (siehe 1.2) folgt die Notwendigkeit für ein System, welches Szeneninformationen analysiert, Nutzer*innenbeschreibungen von Interaktionen verarbeitet und daraus relevante Objekte, Teilobjekte sowie Rollen ableitet, Vivian-kompatible Interaktionsspezifikationen erzeugt und Fehler prüft sowie gezielt korrigiert.

Kapitel 4.

Systemkonzeption und Architekturentscheidungen

Im Folgenden wird der in dieser Arbeit entwickelte Ansatz als *Vivian Specification Generator* (VSG) bezeichnet.

4.1. Anforderungsanalyse

Die Anforderungen an den VSG wurden nicht empirisch durch eine Nutzer*innenstudie erhoben, sondern analytisch aus der Problemstellung, den Forschungsfragen, der Abgrenzung, dem Stand der Forschung sowie den technischen Rahmenbedingungen des Vivian-Frameworks abgeleitet. Im Mittelpunkt stehen die Analyse vorhandener Szenendaten, die Wiederverwendung vorhandener Geometrien, die Erzeugung Vivian-kompatibler Spezifikationen, die niedrigschwellige Eingabe gewünschter Interaktionen sowie die frühzeitige Erkennung und Korrektur syntaktischer, semantischer sowie referenzieller Fehler. Zur Nachvollziehbarkeit wurden die Anforderungen in funktionale Anforderungen (A), nicht-funktionale Anforderungen (N) sowie domänenspezifische Anforderungen (D) unterteilt.

4.1.1. Funktionale Anforderungen

- **A1:** Der VSG soll aus bestehenden Szenenrepräsentationen und natürlichsprachlichen Interaktionsbeschreibungen Vivian-Spezifikationsdateien erzeugen, die alle für die beschriebene Interaktion notwendigen Interaktionselemente, Visualisierungselemente, Zustände und Übergänge enthalten.
- **A2:** Relevante Objekte und Teilobjekte sollen vom VSG explizit identifiziert und Rollen zugeordnet werden, sodass diese eindeutig über Namensreferenzen in den Vivian-Spezifikationen referenziert werden können.
- **A3:** Der VSG soll erzeugte Spezifikationen syntaktisch, semantisch und referenziell prüfen.

- **A4:** Der VSG soll im Generierungsprozess erkannte Fehler in einem begrenzten Korrekturprozess erneut an die zuständigen Verarbeitungsschritte zurückführen.
- **A5:** Der VSG soll einen Human-in-the-Loop-Schritt bereitstellen, in dem Nutzer*innen die Szeneninterpretation überprüfen und gegebenenfalls korrigieren können, bevor daraus Vivian-Spezifikationen erzeugt werden.
- **A6:** Der VSG soll es Nutzer*innen ermöglichen, gewünschte Interaktionen natürlich-sprachlich zu beschreiben, ohne dass Vivian-Spezifikationen manuell erstellt oder bearbeitet werden müssen.

4.1.2. Nicht-funktionale Anforderungen

Diese Anforderungen ergeben sich aus F4 (Fehlerreduktion durch Validierungs- und Selbstkorrekturmechanismen), der Fehleranfälligkeit generativer KI, dem prototypischen Systementwurf sowie der Notwendigkeit, Module und Validierungsregeln später erweitern zu können.

- **N1:** Der VSG soll einzelne Verarbeitungsschritte so kapseln, dass Zwischenartefakte separat validiert und bei erkannten Fehlern gezielt neu erzeugt werden können.
- **N2:** Zwischenartefakte und Entscheidungen des VSG sollen protokolliert und abgespeichert werden, sodass der Generierungs- und Korrekturprozess nachvollzogen werden kann.
- **N3:** Die Implementierung des VSG soll so erweiterbar sein, dass zusätzliche Module ergänzt oder bestehende ersetzt werden können, ohne die Gesamtarchitektur grundlegend zu verändern.

4.1.3. Domänenspezifische Anforderungen

Die domänenspezifischen Anforderungen ergeben sich unmittelbar aus der Verwendung des Vivian-Frameworks. Da Vivian Interaktionsverhalten über Spezifikationsdateien abbildet und die Verbindung zu vorhandenen Szenenobjekten über Namensreferenzen herstellt, sollen diese Anforderungen sicherstellen, dass Spezifikationen den Vorgaben des Frameworks entsprechen und somit Interaktionen mit 3D-Objekten ermöglichen.

- **D1:** Der VSG erzeugt Vivian-Spezifikationsdateien, welche dem von Vivian erwarteten Dateiaufbau, den erlaubten Elementtypen, den Pflichtfeldern sowie zulässigen Wertebereichen entsprechen müssen.

- **D2:** Der VSG darf vorhandene Geometrien und Szenenhierarchien nicht neu generieren oder ersetzen, sondern ausschließlich durch semantische und verhaltensbezogene Spezifikationen ergänzen.
- **D3:** Die erzeugten Bezeichner für Interaktionselemente, Visualisierungselemente, Zustände, Übergänge und Objekt-Referenzen müssen eindeutig sein.
- **D4:** Jede Referenz in Zuständen, Übergängen, Events, Guards und Elementdefinitionen von Interaktionselementen sowie Visualisierungselementen muss auf ein vorhandenes Szenenobjekt oder ein vorhandenes Element der Spezifikation verweisen.
- **D5:** Die generierte Spezifikation gilt als erfolgreich erzeugt, wenn sie in Unity mit dem Vivian Framework importiert und ohne Validierungsfehler ausgeführt werden kann.

4.2. Untersuchte Architekturansätze

Im Laufe dieser Arbeit wurden verschiedene Ansätze implementiert, um das Ziel der Generierung einer vollständigen Spezifikation nach Vivian zu erreichen.

4.2.1. Manager-gesteuerter Multi-Agenten-Ansatz

In der ersten Iteration wurden verschiedene Multi-Agenten-Systeme angeschaut. Im Manager-Agent-getriebenen System kann ein LLM-Agent beliebig viele weitere (spezialisierte) LLM-Agenten als Werkzeuge (Tools) aufrufen, um Aufgaben von spezialisierten Agenten lösen zu lassen, welche dann ein Ergebnis zurück an den Manager liefern. Wichtig zu erwähnen bei diesem Ansatz ist, dass ein Manager Agent spezialisierte Agenten aufrufen kann, aber selbst entscheidet, ob er dies tut. Eine ähnliche Vorgehensweise ist die des dezentralisierten Systems, bei dem es allerdings keine Orchestrierungseinheit wie einen Manager gibt, sondern Agenten die Aufgabe komplett an andere Agenten abgeben können (Handoff), ohne dass diese ein Ergebnis zurück an einen Manager-Agenten liefern.

Um eine übergeordnete Instanz im Prozess zu integrieren, welche sicherstellt, dass Referenzen zwischen den einzelnen Spezifikationsdateien korrekt sind und entstandene Fehler durch gezieltes Aufrufen der zuständigen spezialisierten Agenten korrigieren kann, wurde für die erste Iteration der Manager-getriebene Ansatz gewählt. Als spezialisierte Einheiten wurden ein Szenenanalyse-Agent sowie ein Agent je Spezifikationsdatei gewählt. Weiterhin wurde ein Bestätigungsschritt implementiert, bei dem Benutzer*innen das Ergebnis des Szenenanalyse-Moduls prüfen können.

Im ersten Implementierungsdurchlauf zeigte sich, dass der Manager-getriebene Ansatz besonders bei dateiübergreifenden Referenzen und der konsistenten Benennung von Elementen

zu Fehlern führte. Da der Manager-Agent eigenständig entscheidet, welche untergeordneten Agenten er aufruft, konnten Zwischenergebnisse nicht validiert werden und Fehler sind erst am Ende des Generierungsprozesses aufgefallen. Daher wurde dieser Ansatz zugunsten eines deterministischen modularen Workflows verworfen.

4.2.2. Begründung des modularen Workflow-Ansatzes

Die Architektur des zweiten Implementierungsdurchlaufs folgt dem Ansatz eines modularen Workflows, welcher ermöglicht, dass Module unabhängig beschrieben, validiert und erweitert werden können, ohne die Gesamtarchitektur grundlegend zu verändern (siehe N3). Diese modulare Trennung adressiert N1 und folgt dem in 3 beschriebenen Ansatz von LLMR, welches ebenfalls einen modularen Aufbau verwendet [23]. Jedes der in Abbildung 5.2 aufgeführten Module adressiert eine spezifische Teilaufgabe innerhalb des gesamten Prozesses, wodurch der Informationsfluss gesteuert werden kann und nachvollziehbar bleibt. Der VSG besteht im Wesentlichen aus den Modulen Scene Analyzer, Interaction Planner, Specification Generator, Consistency Reviewer und Unity-Validator.

Bei einfachen LLM-Aufrufen sinkt empirisch die Regelbefolgung bei wachsender Länge und Abhängigkeit strukturierter Ausgaben, sodass halluzinierte Referenzen und unzulässige Elementtypen kaum vermeidbar wären [28]. Ein einzelner LLM-Aufruf wäre somit nicht ausreichend – er müsste fünf referenzierende Spezifikationsdateien gleichzeitig konsistent erzeugen und währenddessen sicherstellen, dass jeder Vivian-Elementtyp die in der Szene tatsächlich vorhandenen Unity-Komponenten besitzt. Durch die Zerlegung in einzelne Schritte sowie den Einsatz von Zwischenvalidierungen und iterativer Fehlerbehebung ermöglicht die hier vorgestellte Pipeline die Erzeugung korrekter Spezifikationen nach Vivian. Eine Skalierung ist durch dieses Vorgehen ebenfalls möglich: Durch Erweiterung bestehender oder Hinzufügen neuer Module könnten noch komplexere 3D-Objekte verarbeitet werden, als in dieser Arbeit betrachtet werden.

Kapitel 5.

Prototypische Implementierung der gewählten Systemarchitektur

In diesem Kapitel wird die Konzeption und prototypische Implementierung des Vivian Specification Generators als gewählte Systemarchitektur zur Spezifikationsgenerierung für das Vivian Framework beschrieben. Ziel war es ein System zu entwerfen, welches die theoretischen Grundlagen und Anforderungen, welche in den vergangenen Kapiteln erarbeitet wurden, in ein lauffähiges und den Anforderungen entsprechendes System umsetzt. Konkret soll dieses semantisch und strukturell korrekte Spezifikationen automatisiert erzeugen, Benutzer*innen-Feedback entgegennehmen sowie aufkommende Fehler iterativ und eigenständig beheben können.

Diese Arbeit ist in Kooperation mit dem Institut für angewandte Informatik (IFAI) der Technischen Hochschule Nürnberg Georg-Simon-Ohm entstanden. Aufgrund dieser Zusammenarbeit sowie der Expertise im 2.3 beschriebenen Vivian Framework, wurde dieses als Grundlage zur Umsetzung einer solchen Pipeline gewählt. Da dieses Framework interaktive Prototypen ausschließlich über eine maschinenlesbare Beschreibung des Verhaltens möglich macht, bietet es eine Grundlage, auf der LLM-gestützte Verfahren aufsetzen können.

5.1. Systemkontext und Schnittstellen

Im Folgenden wird ein Architekturüberblick des VSG gegeben, in dem klar abgegrenzte, spezialisierte Module mit ihrer Funktion im Prozess kurz aufgeführt werden, in welcher Reihenfolge sie aufgerufen werden und welche Informationen jeweils ein und aus gehen.

Zur Interaktion mit dem VSG wird Unity als Benutzer*innenschnittstelle genutzt. In einem Fenster kann eine initiale Interaktionsbeschreibung eingegeben und das zu transformierende statische 3D-Asset ausgewählt werden. Durch die Verwendung vorhandener Geometrie adressiert dieser Schritt D2. Außerdem wird A6 behandelt, da hier natürlichsprachliche Beschreibungen entgegengenommen werden. Abbildung ?? veranschaulicht Unity als Schnittstelle in der Gesamtarchitektur zwischen Benutzer*in und KI-Pipeline.

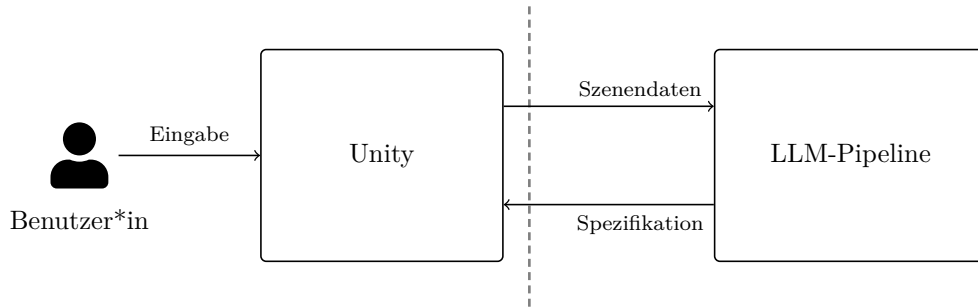


Abbildung 5.1.: Systemkontextdiagramm des VSG.

Unity-Prozesse übersetzen das ausgewählte 3D-Objekt zudem in Formate, die von nachgelagerten Modulen und insbesondere den darin enthaltenen LLMs entgegengenommen werden können und zeigt die geplanten Interaktionen an, damit diese von Benutzer*innen bestätigt oder in einer Iterationsschleife korrigiert werden können. Dieser Human-in-the-Loop-Schritt adressiert A5. Zusätzlich ist Unity die Umgebung, in der Vivian die Anreicherung eines statischen 3D-Assets mit Interaktionslogik über Spezifikationsdateien zur Laufzeit leistet. Nach erfolgreicher Generierung der Vivian-Spezifikation liefert die KI-Pipeline diese zurück an Unity, damit dort Vivian zusammen mit statischem 3D-Asset und dieser Spezifikation ausgeführt werden kann. Diese Gesamtarchitektur adressiert A1, da aus natürlichsprachlichen Eingaben und bestehenden Szenenrepräsentationen vollständige Vivian-Spezifikationen erzeugt werden.

Weiterhin erfolgt die Kommunikation der KI-Pipeline mit Unity über eine REST-API-Schnittstelle, sodass Funktionalitäten der Pipeline gegebenenfalls durch weitere Endpunkte erweiterbar bleiben. Durch diese Entkopplung könnte die KI-Pipeline auch in einen übergeordneten Prozess integriert werden, ohne an die Unity-UI gebunden zu sein.

5.2. Verwendete Sprachmodelle und Agenten

Die im Rahmen dieser Arbeit entwickelte Pipeline wird nicht von einem einzigen, autonomen Agenten gesteuert, sondern innerhalb eines deterministisch orchestrierten Workflows verwendet (siehe 4.2.2). Die Verarbeitungsreihenfolge ist damit überwiegend fest vorgegeben und orientiert sich am modularen Ansatz. Innerhalb solcher Module werden Sprachmodelle von spezialisierten Agenten für klar abgegrenzte Teilaufgaben mit definierten Ein- und Ausgabeformaten. Mit diesem Vorgehen wird der Freiheitsgrad einzelner Modellaufrufe reduziert und die Nachvollziehbarkeit des Datenflusses erhöht. Auch können so fehlerhafte Teilschritte gezielt erneut ausgeführt werden, ohne zwangsläufig den gesamten Prozess neu zu starten.

Die Pipeline des VSG arbeitet mit mehreren Agenten, welche unterschiedliche, teils multimodale Aufgaben zu bewältigen haben. Ein Scene-Analyzer-Agent wird zur semantischen Interpretation einer vorverarbeiteten Szene verwendet und muss dazu sowohl strukturelle JSON-Daten als auch gerenderte Bildansichten eines 3D-Assets verarbeiten. Dieser Agent ist der einzige innerhalb des VSG, welcher mit multimodalen Eingaben arbeitet. Ein Interaction-Planner-Agent überführt dieses Szenenverständnis und die Nutzer*innenbeschreibung der Interaktion in einen strukturierten Interaktionsplan. Das Specification-Generator-Modul überführt mithilfe von vier verschiedenen Agenten den Interaktionsplan in die verschiedenen Teile einer Vivian-Spezifikation. Ein Consistency-Review-Agent überprüft diese generierten Einzelartefakte auf dateiübergreifende sowie semantische Konsistenz. Sollten Fehler im Generierungsprozess entstanden sein, leitet ein Fixer-Agent Korrekturmaßnahmen aus diesen ab. Die verwendeten Agenten arbeiten somit als spezialisierte Einheiten innerhalb eines kontrollierten Gesamtsystems.

Anstelle von Freitext generieren die Agenten strukturierte Ausgaben. Im Rahmen dieser Arbeit wurden überwiegend „Pydantic“-Modelle als Ausgabetypen (*output_type*) festgelegt, sodass das verwendete Sprachmodell in genau diesem Schema antwortet. Dadurch sind Ergebnisse direkt maschinenlesbar und können gegebenenfalls weiterverarbeitet, validiert, gespeichert oder an nachfolgende Module überreicht werden. Diese Einschränkung der Antwortvariabilität verbessert die Kontrollierbarkeit des gesamten Systems. Freitext wird dennoch dort zugelassen, wo Begründungen, Korrekturhinweise oder Zusammenfassungen angegeben werden sollen.

Die Wahl der verwendeten LLMs erfolgt je nach Aufgabe und orientiert sich an Reasoning-Bedarf, Kontextumfang sowie Kosten. Stärkere Modelle werden dort eingesetzt, wo hohe Interpretations- und Planungsleistung benötigt wird, während schemagebundenere Aufgaben mit klaren Anweisungen und kleineren Modellen umgesetzt werden können. Der VSG nutzt bewusst nicht für jeden Agenten dasselbe Sprachmodell, sondern passt die Auswahl an die jeweilige Aufgabe des Agenten an. Trotz dieser Spezialisierungen bleiben bekannte Grenzen LLM-basierter Verfahren, wie Halluzinationen, semantische Fehlannahmen und fehlerhafte Referenzen, bestehen, weshalb der Einsatz von Agenten in der vorliegenden Arbeit durch Human-in-the-Loop, Validierungsmechanismen und iterative Korrekturschritte abgesichert wird. Diese Absicherung adressiert A3 (Validierung der Spezifikation), A5 (Human-in-the-Loop) sowie F4 (Fehlerreduktion durch Validierungs- und Selbstkorrekturmechanismen).

Die nachfolgenden Abschnitte beschreiben die einzelnen Module und ihre Aufgaben innerhalb der Pipeline. Bei Verarbeitungsschritten, welche LLM-basierte Agenten verwenden, wird zusätzlich der Einsatz verwendeter Sprachmodelle erläutert.

5.3. Gesamtpipeline und Datenfluss

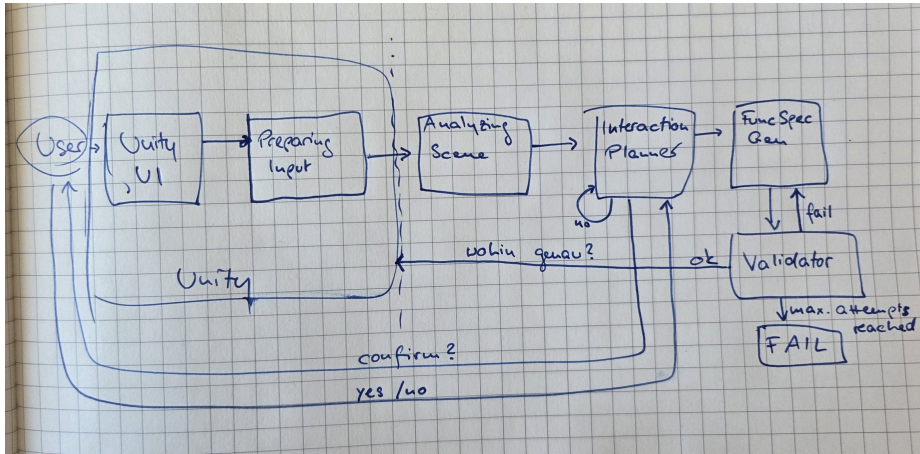


Abbildung 5.2.: Container-Diagramm des VSG.

Ein*e Benutzer*in kann über die Unity UI mit dem System interagieren und erhält Informationen über den Prozessfortschritt, Logging-Informationen, Rückmeldung des Interaktionsplaners und ob ein Durchlauf erfolgreich war. Anschließend wird das ausgewählte 3D-Asset über ein weiteres Unity-Skript in eine strukturelle Szenenrepräsentation umgewandelt sowie ergänzende 2D-Bildansichten gerendert.

Diese aufbereitete Szenendarstellung wird im darauffolgenden Schritt zusammen mit einer initialen Benutzer*innenbeschreibung der gewünschten Interaktivität an den Scene Analyzer übermittelt. Dieser gibt ein Scene-Understanding-Objekt aus, welches für jedes relevante Objekt dessen Funktion, Rolle und Eignung als potenzielles Interaktions- oder Visualisierungselement enthält.

Anschließend erhält der Interaction Planner das zuvor generierte Scene-Understanding-Objekt zusammen mit der strukturierten Szenenrepräsentation, die zuvor auch der Scene Analyzer erhalten hat, und der natürlichsprachlichen Interaktionsbeschreibung. In diesem Schritt wird geplant, welche Interaktions- und Visualisierungselemente es geben wird, welche Zustände der Prototyp einnehmen kann und welche Übergänge zwischen Zuständen vorgesehen sind. Der Interaction Planner gibt einen Interaction-Plan aus, welcher Elementrollen, geplante Zustände und Übergänge sowie eine Liste der zu generierenden Spezifikationsdateien enthält. Dieser Plan wird gemäß A5 der Benutzer*in anschließend zur Bestätigung oder Korrektur vorgelegt. Bei einer Korrektur wird der Interaction Planner erneut aufgerufen, um diese anzuwenden, bis der Plan bestätigt wird, welcher schließlich den nachgelagerten Modulen als verbindliche Vorgabe dient.

Der Specification Generator erzeugt auf Grundlage dieses Interaktionsplans die eigentlichen Vivian-Spezifikationsdateien in der notwendigen Reihenfolge. Ausgegeben werden vier strukturierte JSON-Dateien, welche zusammen eine Vivian-Spezifikation bilden.

Der Consistency Reviewer prüft nun mithilfe des Interaktionsplans, ob der Plan korrekt vom Specification Generator umgesetzt wurde und ob logische Widersprüche oder dateiübergreifende Inkonsistenzen existieren. Das könnten z.B. unerreichbare Zustände oder widersprüchliche Bedingungen sein, welche eine rein strukturelle Validierung nicht erkennen kann

Ein Validator ergänzt dies um eine strukturelle und laufzeitnahe Kontrolle. Wird die maximale Anzahl an Versuchen erreicht oder keine Fehler gefunden, terminiert der VSG. Letzteres sorgt für ein Zurückliefern der generierten Vivian-Spezifikation an Unity, sodass Vivian diese zusammen mit dem statischen 3D-Asset ausführen kann.

5.4. Vorverarbeitung in Unity

Um ein 3D-Asset in MLLM-verständliche Daten umzuwandeln, müssen eine strukturelle Repräsentation und Ansichten aus verschiedenen Blickwinkeln erstellt werden. Da der nachgelagerte Scene Analyzer mit Text und Bild als Eingaben arbeitet, benötigt er die erstellten Ansichten als visuelles Grounding für die abstrakten JSON-Daten. Die Form des Objekts, das Material oder auch räumliche Anordnung werden somit direkt sichtbar und müssen nicht aus Bounding-Boxen rekonstruiert werden.

Im ersten Schritt wird dazu aus dem im Unity-Fenster (Abbildung 5.3) ausgewählten 3D-Asset ein *Prefab*-Objekt erzeugt, welches später von Vivian zur Laufzeit instanziiert wird. Damit wird gemäß D2 keine neue Geometrie generiert, sondern wiederverwendet.

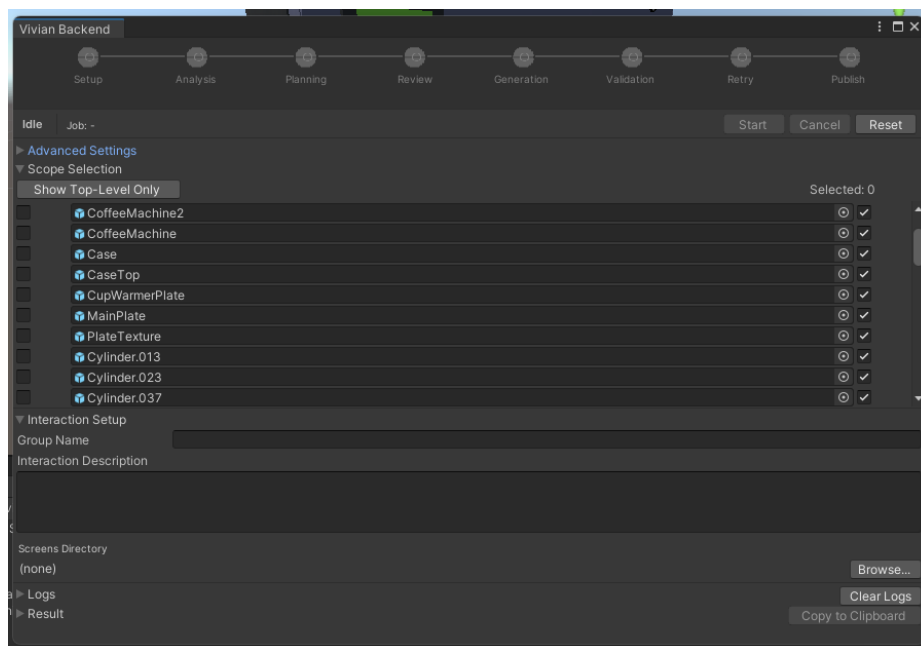


Abbildung 5.3.: Unity UI des VSG.

Um einem LLM Verständnis über ein 3D-Asset (in Unity „GameObject“ genannt) zu geben wird zunächst eine strukturierte Szenenbeschreibung im JSON-Format mit dem Namen *scene.json* erzeugt. Dazu wird die Hierarchie des Szenengraphen rekursiv traversiert und pro GameObject Informationen wie die ID, der Hierarchiepfad, Transformation, Materialien, Bounding Boxes oder Mesh-Statistiken extrahiert. Weiterhin werden bereits basierend auf Namen, Tags oder Material-Hinweisen heuristisch Rollen erkannt, welche im späteren Verlauf verwendet werden können, um Interaktionselemente und Visualisierungselemente zu klassifizieren.

Im Anschluss daran rendert Unity für das übergeordnete GameObject verschiedene Ansichten, da die Erkennungsleistung eines MLLM steigt, wenn statt einer Einzelansicht mehrere gerenderte Ansichten eines 3D-Objekts verwendet werden [29]. Dafür werden sechs orthografische Ansichten entlang der Hauptachsen sowie zwei isometrische (von schräg oben) als 1024x1024-Pixel-PNGs erstellt (siehe Abbildung 5.4). Die gewählte Anzahl der Ansichten ist ein Kompromiss zwischen Erkennungsrobustheit und minimal notwendigem Bildumfang, um Kosten pro Durchlauf zu sparen. Die orthografischen Ansichten vermeiden perspektivische Verzerrung und beinhalten saubere Silhouetten, während die isometrischen Ansichten die räumliche Gesamtform und Lesbarkeit verbessern sowie drei Raumdimensionen in einer Darstellung erfassbar machen. Zur Laufzeit wird dazu eine Kamera angelegt, anhand der Welt-Bounding-Box des Objekts platziert und auf das Objektzentrum ausgerichtet. Alle dafür benötigten Ressourcen werden anschließend wieder freigegeben.

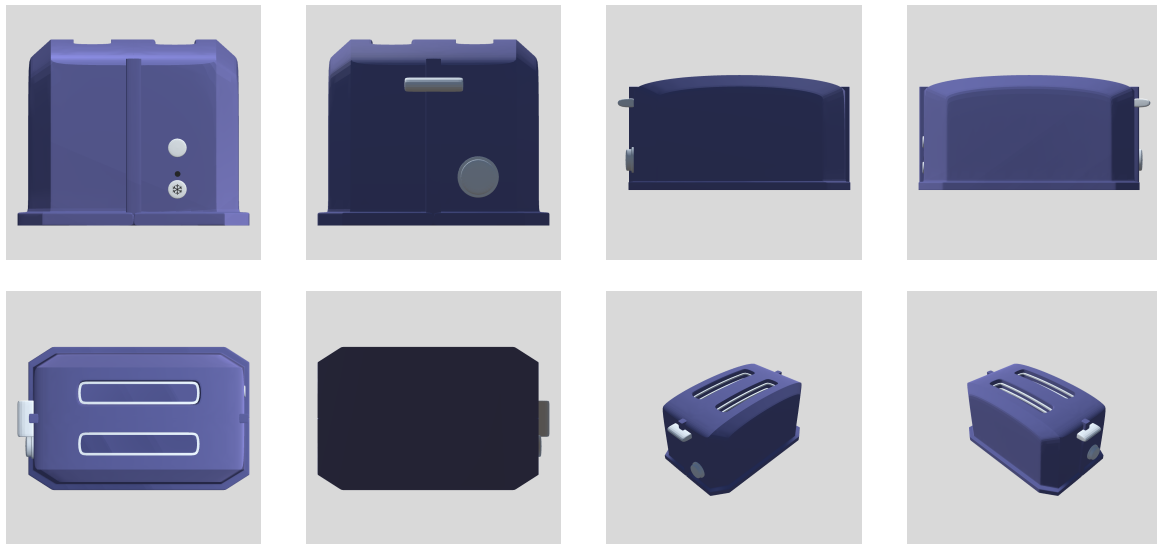


Abbildung 5.4.: Acht gerenderte Ansichten eines Toaster-3D-Assets.

Eine zugehörige Datei namens *views_manifest.json* enthält pro Bild Kameramatrizen, Pose, Projektionstyp und einen Vermerk, welches Teilobjekt des Szenengraphen sich an welcher Stelle in der Ansicht befindet. Unity kennt die Kameraposition die 3D-Box jedes Objekts in der Szene und kann so ausrechnen, wo sich ein Objekt der Szene im jeweiligen Bild befindet.

Nachgelagerte MLLMs können so den Zusammenhang zwischen Objekten in *scene.json* und denen aus den Ansichten (siehe Abbildung 5.4) herstellen. Dafür wird die ID, ein Tiefenhinweis und das Pixelrechteck vermerkt, in dem sich das Objekt im Bild befindet.

Der Vorverarbeitungsschritt in Unity erzeugt also zusammengefasst aus einem vorhandenen, statischen 3D-Asset eine strukturelle Repräsentation als JSON-Struktur, acht gerenderte Ansichten von diesem sowie eine weitere JSON-Struktur mit Informationen zu den Ansichten. Diese Daten können nun vom nächsten Modul, dem Scene Analyzer, konsumiert werden. In diesem Schritt werden D2, D4 sowie N1 adressiert, da Geometrie erhalten bleibt, die JSON-Struktur eine Basis für spätere Namensreferenzen bietet und definierte Ein- und Ausgabeformate verwendet werden.

5.5. Scene Analyzer

Die Kernaufgabe des Scene-Analyzer-Moduls besteht darin, die in der Vorverarbeitung entstandenen visuellen Eindrücke und strukturellen Daten mit der natürlichsprachlichen Nutzer*innenbeschreibung zu verknüpfen und die rohen Daten so in ein semantisch angereichertes Zwischenformat zu überführen. F2 wird hier adressiert, da relevante Objekte und Teilobjekte aus Szeneninformationen und der Beschreibung abgeleitet werden, sodass nachfolgende Schritte auf diese referenzieren können.

Als Eingaben bekommt der Scene Analyzer die in Abschnitt 5.4 beschriebene strukturierte Szenenrepräsentation und die Ansichten des Objekts. Die JSON-Struktur stellt Namen, Pfade, IDs und Geometrie bereit, während die Bilder bei der semantischen Einordnung helfen. Diese Daten werden von einem internen LLM-basierten Agenten verarbeitet, welcher so einordnen kann, ob ein Teilobjekt beispielsweise eher Griff, Display oder Knopf ist.

Das verwendete LLM leistet in diesem Modul die Hauptarbeit: Es klassifiziert die Szeneninformationen gemäß einem vorgegebenen Schema, dem *SceneUnderstanding*. In Anhang A.1 ist dargestellt, wie der verwendete Agent mithilfe eines strukturierten Prompts angesteuert wird.

Dieses *SceneUnderstanding* beinhaltet nun ein aufbereitetes Szenenverständnis mit Informationen zu Objektbezügen, Relationen oder Clustern und bietet die Grundlage dafür, dass darauffolgende Agenten ein Verhalten aus der analysierten Szene ableiten können. Durch dieses Zwischenprodukt muss der Interaction Planner nicht direkt aus einer Unity-Hierarchie schließen, welche Objekte funktional relevant sind, sondern kann mit einer semantisch vorstrukturierten Repräsentation arbeiten.

Diese entstandene kontrollierte Zwischenrepräsentation ist maschinenlesbar und validierbar, bleibt allerdings dennoch von der Qualität der Modellinterpretation abhängig und ist anfällig für semantische Falschannahmen, weshalb spätere Review-Schritte relevant bleiben.

5.6. Interaction Planner

Der Interaction Planner bildet die Brücke zwischen Szenenanalyse und Spezifikationsgenerierung. Die semantisch verstandene Szenenrepräsentation wird in einen strukturellen Plan überführt, nach dem der nachgelagerte Generierungsschritt die Spezifikationsdateien erzeugt. Da das in der Szenenanalyse generierte *SceneUnderstanding* Objekte rein deskriptiv interpretiert, werden diese im Interaktionsplanungsschritt nach Vivian-Functional-Specification-Typen klassifiziert und damit konkreten Vivian-Rollen zugeordnet. So wird außerdem vermieden, dass einzelne Agenten in der Spezifikationsgenerierung inkonsistente Entscheidungen treffen. Er ist auf die Strukturen und Regeln des Vivian Frameworks ausgerichtet und kann so einzelnen Teilobjekten bestimmte Rollen zuweisen, Zustände und Übergänge planen sowie entscheiden, welche Dateien der Spezifikation für die geplante Interaktion benötigt werden. Weiterhin wurde ein Human-in-the-Loop-Schritt implementiert, damit Nutzer*innen die Ausgabe des Interaction Planners validieren oder korrigieren können.

Dafür wird erneut ein LLM-Agent verwendet, welcher bestimmte Instruktionen und somit Informationen über das Vivian Framework erhält (siehe A.2). Mit diesem Wissen kann er die beschriebenen Aufgaben zuverlässig bewältigen. Als LLM wird hier das aktuell leistungsstärkste Modell von OpenAI, GPT-5.5, verwendet, da der Agent aus den Rohdaten der Szene ein Interaktionsverhalten nach Vivian herleiten muss. Dabei können schwerwiegende Fehler entstehen wie falsche Objekttypen oder Halluzinationen. Solche Fehler sind schwerwiegend, da der Interaction Planner eine zentrale Planungseinheit für den späteren Prototypen darstellt und sie in nachfolgende Schritte weitergereicht werden. Somit würden auch alle nachgelagerten Schritte fehlerhafte Ausgaben produzieren.

Das vom Scene Analyzer ausgegebene *SceneUnderstanding*-Objekt (siehe 5.5) im JSON-Format sowie die vom Unity UI weitergereichte natürlichsprachliche Interaktionsbeschreibung werden dazu vom Interaction Planner konsumiert und in einen strukturierten Interaktionsplan (*InteractionPlan*) überführt. Dieser enthält ein vordefiniertes Format, welches Informationen über Elementrollen, geplante Zustände und Übergänge sowie eine Liste der benötigten Spezifikationsdateien enthält. Listing 5.1 zeigt einen Strukturüberblick eines Beispiel-Interaktionsplans. Darüber hinaus liefert das LLM noch eine zusammengefasste Begründung für die getroffenen Entscheidungen mit, welche es Nutzenden erleichtern soll Fehler zu finden und zu beheben (siehe 5.1, Z. 6).


```

1 {
2   "element_roles": [...],
3   "planned_states": [...],
4   "planned_transitions": [...],
5   "files_needed": [...],
6   "reasoning": "The user description was treated as the
   primary source, so the plan focuses only on the
   handle-triggered toast cycle and the two requested
   visual feedback elements. Handle is modeled as a
   Slider because the scene confirms linear y-axis
   travel, while the heaters and LED are modeled as
   Light elements to support on/off glowing behavior
   during toasting. Other controls such as the rotary
   knob and buttons were excluded because they were not
   requested and are not necessary for the described
   prototype behavior."
7 }

```

Listing 5.1: Beispiel eines Interaktionsplans

Eine der Hauptaufgaben des Interaction Planners ist die Rollenzuweisung. Dafür werden den relevanten Objekten einer 3D-Szene basierend auf Namen, Materialien und Geometrie spezifische Vivian-Typen zugewiesen, wie *Button*, *Slider*, *Rotatable*, *Light* oder *Screen*, welche das LLM direkt aus den einzelnen Elementen von *SceneUnderstanding* ableitet. Beispielsweise sollte einem *Handle* die Rolle eines *Slider* zugewiesen werden. Diese Aufgabe adressiert A2 (Identifikation und Rollenzuweisung relevanter Objekte und Teilobjekte)

Weiterhin definiert der Interaction Planner Zustände, welche der spätere Prototyp einnehmen kann sowie Übergänge zwischen diesen. Das umfasst das Erstellen von Zustandsnamen, Beschreibungen, Übergangsnamen und möglichen Auslösern (*Triggern*), die diese Übergänge initiieren. In Listing 5.2 ist ein beispielhafter Ausschnitt eines Interaktionsplans mit einem geplanten Übergang zu sehen. Die Zustände und Übergänge werden hier bereits geordnet beschrieben, allerdings erst im Functional-Specification-Generator-Schritt (siehe 5.7) in Spezifikationsdateien umgewandelt. Außerdem kann das LLM festlegen, an welche Bedingungen (*Guard Hints*) ein Übergang geknüpft sein kann (siehe 5.2, Z. 7–8).

```

1 {...
2   "planned_transitions": [{
3     "source_state": "idle.state",
4     "destination_state": "toasting.state",

```

```

5      "trigger_element": "Handle",
6      "trigger_description": "User pushes the handle down
      to start toasting.",
7      "guard_hints": [
8          "Handle VALUE > 0.8"
9      ]}]...]}

```

Listing 5.2: Beispiel eines Interaktionsplans: geplanter Übergang

Eine weitere Aufgabe des Interaction Planners umfasst die Bestimmung des Umfangs, also welche konkreten Spezifikationsdateien (*InteractionElements.json*, *VisualizationElements.json*, *States.json* und *Transitions.json*) für die gegebene Szene erforderlich sind, da dies von der Komplexität des Prototyps abhängt. Einfachere Prototypen mit vergleichsweise wenigen Elementen oder Funktionen benötigen gegebenenfalls nicht alle Dateien einer Spezifikation, während komplexere alle vier Dateien benötigen können. Durch modulare Kapselung der nachfolgenden Schritte (N1) können gegebenenfalls einzelne Generierungsagenten in 5.7 übersprungen werden, sodass nicht benötigte Spezifikationsdateien nicht erzeugt werden.

Der generierte Interaktionsplan wird anschließend zurück an die Unity UI gesendet und den Benutzer*innen zur Prüfung angezeigt. Über die Entscheidungsbegründung kann direkt ein erster Eindruck gewonnen werden, ob Interaktionen wie gefordert geplant worden sind. Bei Bedarf lassen sich zudem einzelne Interaktionselemente, Visualisierungselemente, Zustände und Übergänge anzeigen, um diese bei Fehlern auch einzeln adressieren zu können.

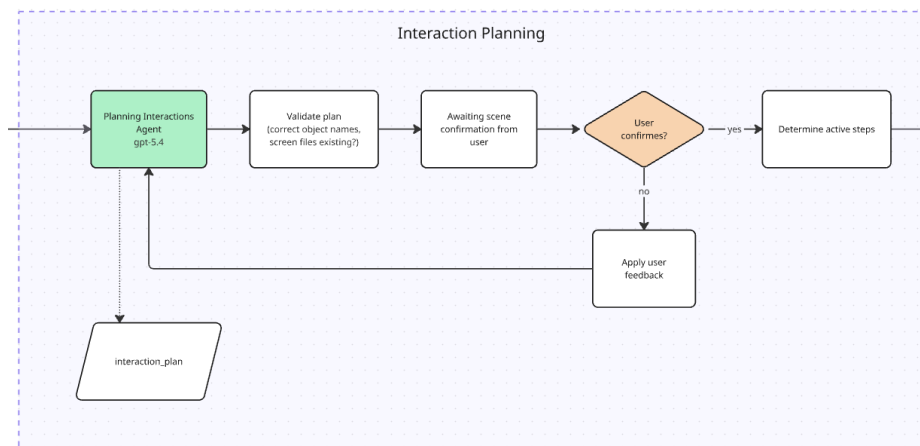


Abbildung 5.5.: Interaction Planner und Human-in-the-Loop.

Sollte eine Korrektur nötig sein, können Nutzende über ein Freitextfeld in natürlicher Sprache beschreiben, welche Interaktionen oder Elemente geändert werden sollen. Der Agent zur Interaktionsplanung wird daraufhin erneut ausgeführt und nimmt Änderungen am Interaktionsplan vor. Dieser in Abbildung 5.5 abgebildete iterative Prozess stellt sicher, dass Be-

nutzer*innenrückmeldungen berücksichtigt werden und adressiert A5 (Human-in-the-Loop) sowie F1 (Co-Creation-Workflow).

Die in diesem Schritt implementierte Extraktion komplexer Interaktionslogiken durch Planung von Elementrollen sowie Zuständen und Übergängen mithilfe natürlicher Sprache adressiert direkt F3 und erzeugt damit die zentrale Planungsgrundlage für nachfolgende Module, korrekte Spezifikationsdateien nach Vivian zu erzeugen. Da Fehler an die gesamte Spezifikationsgenerierung weitergereicht werden, wirkt der Human-in-the-Loop-Schritt direkt entgegen, in dem Nutzer*innen den Plan vor der Weitergabe an den Specification Generator bestätigen oder korrigieren können.

5.7. Specification Generator

Der Specification Generator übersetzt die im vorherigen Schritt geplante Interaktionslogik aus einem deklarativen und zum Teil noch natürlichsprachlich beschriebenen Plan in die vier JSON-Dateien einer Vivian-Spezifikation. Während andere Module für das Planen (siehe 5.6) sowie die Validierung der erzeugten Dateien (siehe 5.9) zuständig sind, ist dies der einzige Schritt, in dem tatsächliche, von Vivian konsumierbare JSON-Spezifikationsdateien erzeugt werden. Dieses Modul adressiert damit primär A1 (Erzeugung von Vivian-Spezifikationsdateien) sowie D1 (Erzeugte Dateien müssen den Anforderungen von Vivian entsprechen).

Eingaben, Ausgaben und Verarbeitungsreihenfolge

Als Eingabe nimmt der Specification Generator einen bestätigten Interaktionsplan entgegen, und gibt die Spezifikationsdateien *InteractionElements.json*, *VisualizationElements.json*, *States.json* und *Transitions.json* aus. Da Vivian außerdem eine fünfte Datei (*VisualizationArrays.json*) zur Ausführung benötigt, wird diese ebenfalls erstellt und ist standardmäßig ungefüllt. Da diese fünfte Datei im Kontext dieser Arbeit keine weitere Relevanz hat bleibt sie im Rest der Arbeit unerwähnt. Eine Vivian-Spezifikation gilt somit in dieser Arbeit als vollständig, wenn sie *InteractionElements.json*, *VisualizationElements.json*, *States.json* und *Transitions.json* enthält.

Für die Umsetzung dieses Moduls wurden vier spezialisierte LLM-Agenten implementiert, welche nacheinander aufgerufen werden, um je eine der vier Spezifikationsdateien zu generieren. Zuerst werden Interaktionselemente, danach Visualisierungselemente, anschließend Zustände und im letzten Schritt Übergänge erstellt (siehe 5.7). Die Reihenfolge der Generierung ergibt sich aus den Datenabhängigkeiten: Zustände referenzieren die Namen der Interaktions- und Visualisierungselemente, Übergänge wiederum sowohl Namen der Interaktions- und Visualisierungselemente als auch der Zustände. Eine parallele Verarbeitung würde zwangsläufig zu Inkonsistenzen in den Bezeichnern und somit später zu Fehlern

in der Ausführung führen. Die jeweils generierten Daten werden noch nicht in finalen JSON-Dateien, sondern in einem *Registry*-Objekt gespeichert, welches als *Single-Source-of-Truth* agiert. Dies ermöglicht Datenkonsistenz und verhindert Redundanz. Die verwendeten Agenten sind gegenüber einem einzigen Sprachmodell bei vielen Referenzen zwischen generierten Daten und strukturierten Ausgaben empirisch weniger fehleranfällig, wie bereits in 4.2.2 erwähnt. Diese Aufteilung adressiert zudem N1 (modulare Kapselung und gezielte Neugenerierung) und ermöglicht zudem eine gezielte Wiederausführung einzelner Schritte, ohne sämtliche Spezifikationsdaten neu zu generieren. Abbildung 5.6 zeigt diesen sequenziellen Ablauf inklusive zwischengeschalteter Validierungsstufen.

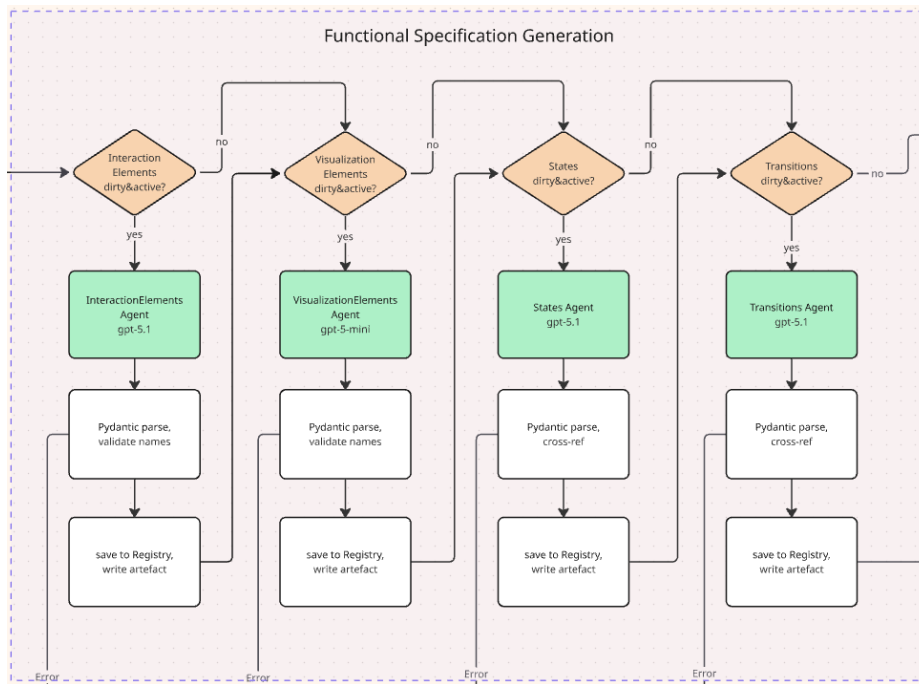


Abbildung 5.6.: Functional Specification Generator.

Modellwahl und Spezialisierung der Agenten

Wie bereits in 2.4 erläutert, wurden auch in diesem Schritt die verwendeten Sprachmodelle je nach Aufgabenkomplexität gewählt. Während der Interaction Planner mehrdeutige Rohdaten als Eingaben erhält, daraus einen für Folgeschritte verbindlichen Plan entwickelt (siehe 5.6) und deshalb das stärkste aktuell verfügbare Modell verwendet, arbeiten die Agenten der Specification Generation mit einer vorstrukturierten, von Benutzenden bestätigten Grundlage. Daher können die Agenten verhältnismäßig kleinere und schnellere LLMs verwenden. Tabelle 5.1 zeigt eine Übersicht der im Specification Generator verwendeten Sprachmodelle je Agent.

Agent	Sprachmodell
InteractionElements	gpt-5.1-2025-11-13
VisualizationElements	gpt-5.1-2025-11-13
States	gpt-5.5-2026-04-23
Transitions	gpt-5.1-2025-11-13

Tabelle 5.1.: Übersicht der im Specification Generator verwendeten Sprachmodelle.

Die *InteractionElements*- und *VisualizationElements*-Agenten haben die primäre Aufgabe eine schematische Struktur abzubilden und verwenden daher ein kleineres und kosteneffizienteres Modell (GPT-5.1) als der Interaction Planner. Je Elementtyp gibt es feste Felder, normalisierte Wertebereiche und verpflichtende *Constraints*, sodass die Generierung der jeweiligen Spezifikation im Wesentlichen daraus besteht, bereits vorhandene Szeneninformationen, wie Geometrie oder Achsen, auf das Vivian-Spezifikationsschema abzubilden. Listing 5.3 zeigt einen für den *InteractionElements*-Agenten relevanten Ausschnitt des Interaktionsplans. Schlussfolgerungen (*Reasoning*) werden lediglich dort gefordert, wo Mehrdeutigkeiten bestehen (z. B. *Button* oder *ToggleButton*), um diese aufzulösen. Komplexeres, mehrstufiges *Reasoning* wird an dieser Stelle nicht benötigt, daher wäre ein leistungsstärkeres Modell wie GPT-5.5 bei diesen Agenten überdimensioniert. Die Verwendung eines kleineren Modells, wie etwa GPT-5-mini, hat hingegen häufiger zu Reparaturzyklen geführt und dadurch insgesamt höhere Kosten verursacht.

```

1 {...
2   "element_roles": [
3     {
4       "object_name": "Handle",
5       "funspec_type": "Slider",
6       "category": "interaction",
7       "rationale": "User requested that pushing the
          toaster handle down starts toasting; the scene
          provides y-axis translation parameters for this
          handle.",
8       "interaction_params": {
9         "axis": "y",
10        "range": 0.05754443258047104
11      }
12 }...]}

```

Listing 5.3: Beispiel eines Interaktionsplans: Slider-Elementrolle

Der *States-Agent* verwendet mit GPT-5.5 das leistungsstärkste Modell der hier verwendeten, da die Generierung einer funktionierenden *State-Machine* eine deutlich logikintensivere Aufgabe darstellt als die anderen Bereiche des Specification Generators. Dabei muss der Agent eine Menge von Zuständen erstellen, welche logisch zusammenpassen und das gesamte Verhalten des Prototyps beschreiben. Er muss diese nicht nur benennen, sondern zusätzlich mit passenden und typisierten Bedingungen (*Conditions*) versehen. Ein Beispiel einer *Condition* aus *states.json* ist in Listing 5.4 zu sehen. Werden Zustände von diesem Agenten falsch definiert oder uneinheitlich beschrieben, propagieren diese Fehler unmittelbar in die nachgelagerten Verarbeitungsschritte. Das verbraucht wiederum insgesamt mehr Ressourcen, da diese Fehler in zusätzlichen Reparaturzyklen behandelt werden müssen. Je besser in diesem Schritt Zusammenhänge erkannt und konsistente Zustände erzeugt werden können, desto geringer ist die Wahrscheinlichkeit für fehlerhafte Ausgaben. Deshalb profitiert dieser Agent von einem Modell mit höherer Reasoning-Qualität.

```

1 {
2   "States": [
3     {"Name": "toasting.state",
4       "Conditions": [
5         {
6           "Type": "InteractionElementCondition",
7           "InteractionElement": "Handle",
8           "Attribute": "VALUE",
9           "Value": "1.0"
10        }...]}...]}

```

Listing 5.4: Beispiel einer Zustandsbedingung aus *states.json*

Der *Transitions-Agent* verwendet wie die *InteractionElements-* und *VisualizationElements-* Agenten das Modell GPT-5.1, da die zu verwendenden Zustände bereits im *States-Agent* generiert wurden und hier verwendet werden. Diese Aufgabe benötigt somit verhältnismäßig weniger Reasoning-Aufwand. Grundsätzlich würde dieser Agent zwar aufgrund bestimmter Aufgaben (wie Exklusiv-Oder-Gatter) von einem leistungsstärkeren Modell profitieren, allerdings reicht GPT-5.1 hier erfahrungsgemäß aus. Sollten deutlich komplexere Prototypen als im Rahmen dieser Arbeit betrachtet werden, würde der VSG in diesem Schritt unmittelbar von der Verwendung eines leistungsstärkeren Modells, wie GPT-5.2 oder GPT-5.5, profitieren.

Datenfluss zwischen den Agenten

Als Eingabe erhalten die vier Spezifikationsagenten den Interaktionsplan, sowie bereits generierte Teile der *Registry*. Die Agenten erhalten dabei nicht die gesamten, in der *Registry* gespeicherten Daten, sondern ausschließlich die für ihre eigene Aufgabe benötigten Informationen wie Objektnamen oder -typen. In Tabelle 5.2 ist aufgeführt, welche Eingaben die jeweiligen Spezifikationsagenten bekommen. Diese Informationen werden benötigt, um korrekte Referenzen zwischen den einzelnen Spezifikationsteilen zu erzielen. Der *Transitions*-Agenten bekommt hier bewusst keine Daten der Visualisierungselemente, da für Übergänge ausschließlich Interaktionselemente als *Trigger* fungieren können und diese nicht zur Generierung von Übergängen benötigt werden. Von Zuständen werden weiterhin ausschließlich Namen benötigt (um Quell- und Zielzustand festzulegen).

Agent	Eingabe
InteractionElements	InteractionPlan
VisualizationElements	InteractionPlan, InteractionElements-Subset
States	InteractionPlan, InteractionElements-Subset, VisualizationElements-Subset
Transitions	InteractionPlan, InteractionElements-Subset, States-Subset

Tabelle 5.2.: Eingaben der Spezifikationsagenten.

Validierung und Registry

Jeder der Agenten liefert ein *Pydantic*-typisiertes Objekt zurück. Wie bereits in 5.5, wird hierfür das Argument *output_type* der OpenAI-Agenten verwendet. Die Agenten produzieren somit direkt Objekte im jeweils benötigten Schema. Anschließend durchläuft die generierte Ausgabe zwei Validierungsstufen. Zunächst prüft eine *Pydantic*-Schemavalidierung Pflichtfelder, Wertebereiche und erlaubte Elementtypen. Anschließend werden Querverweise gegen den Interaktionsplan und das bisher gefüllte *Registry*-Objekt überprüft und somit sichergestellt, dass alle generierten Bezeichner auf existierende Szenenobjekte bzw. bereits erstellte Spezifikationselemente verweisen. Nach erfolgreicher Validierung wird das Ergebnis eines Agenten in die *Registry* übernommen und damit für nachfolgende Spezifikationsagenten und Validierungen innerhalb dieses Moduls als *Single-Source-of-Truth* zur Verfügung gestellt. Schlägt eine dieser Prüfungen fehl, startet ein neuer Generierungsversuch. Diese Validierungsschichten adressieren zusammen mit den Agenteninstruktionen D1 (korrekte Vivian-Spezifikationsdateien), D3 (eindeutige Bezeichner) sowie D4 (Vivian-Bezeichner verweisen auf vorhandene Szenenobjekte).

Korrekturprozess

Sollten in nachgelagerten Prüfschichten außerhalb dieses Moduls, wie dem 5.8 oder 5.9, Inkonsistenzen oder Fehler aufkommen, wird ein erneuter Versuch gestartet, die Spezifikation zu erzeugen. Dabei wird nicht zwangsläufig die gesamte Spezifikation, sondern gezielt nur die Teile neu generiert, welche betroffen sind. Dazu werden betroffene Schritte innerhalb des Specification Generators sowie alle davon abhängigen als „dirty“ markiert und ausschließlich diese Teilmenge in einem neuen Versuch durchlaufen. Sollte beispielsweise die *Transitions.json*-Datei Fehler enthalten, wird ausschließlich der *Transitions*-Agent erneut aufgerufen. Wenn hingegen die *InteractionElements*-Datei fehlerhaft ist, müssen alle vier Agenten erneut ausgeführt werden. Dieses Verhalten adressiert A4 (begrenzter Korrekturprozess im Generierungsprozess). Die Abhängigkeiten der Spezifikationsdateien sind 5.7 zu entnehmen.

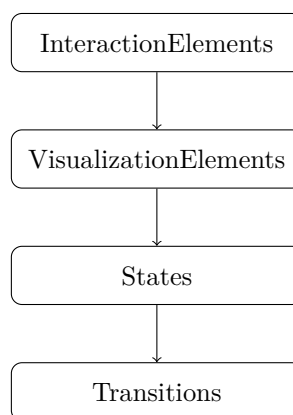


Abbildung 5.7.: Abhängigkeitsgraph der generierten Spezifikationsdateien.

Diese Wiederausführung einzelner Schritte im Generierungsprozess der Spezifikation wird durch die Persistenz des *Registry*-Objekts ermöglicht. Da fehlerfreie Spezifikationsteile über mehrere Versuchsdurchläufe hinweg erhalten bleiben, können die zugehörigen Generierungsschritte übersprungen werden. Weiterhin werden entstandene Zwischenartefakte (wie Spezifikationsdateien) je Generierungsversuch in einem Verzeichnis abgelegt. Dazu wird je Durchlauf ein Protokoll mit sämtlichen vorgenommenen Änderungen am *Registry*-Objekt zusammen mit den dazugehörigen Spezifikationsdateien geführt, um die Veränderungen über alle Versuche eines Durchlaufs hinweg festzuhalten. So können Fehlerursachen besser nachvollzogen und einzelne Durchläufe reproduziert werden. Damit wird N2 (Protokollierung der Zwischenartefakte und Entscheidungen) adressiert.

5.8. Consistency Reviewer

Nachdem nun alle Daten einer Vivian-Spezifikation vom Specification Generator inklusive korrekter Abhängigkeiten und Querverweise generiert wurden, muss im nächsten Schritt geprüft werden, ob semantische Inkonsistenzen in diesen existieren. Dieser Schritt ist notwendig und wichtig, da rein strukturelle Validatoren (JSON-Schema, Unity-Validator) nicht jede Art von Fehler erkennen können. Er prüft die inhaltliche Konsistenz der einzelnen Teilspezifikationen miteinander (ob die Registry-Subsets der Zustände und Übergänge semantisch zu denen der Interaktionselemente sowie Visualisierungselemente passen) sowie dem ursprünglichen Interaktionsplan (ob sämtliches im Interaktionsplan geplantes Verhalten tatsächlich so wie dort aufgeführt umgesetzt wurde). Letzteres beinhaltet Vivian-Typzuweisungen einzelner Interaktions- und Visualisierungselemente sowie die Umsetzung geplanter Zustände und Übergänge.

Um diese Aufgaben bewältigen zu können bekommt der Consistency Reviewer einen User-Prompt bestehend aus dem kompletten Registry-Objekt (*REGISTRY_JSON*) und dem Interaktionsplan (*INTERACTION_PLAN_JSON*) sowie einen System-Prompt, in dem Instruktionen mit den zu erfüllenden Aufgaben enthalten sind. Eine volle Version des System-Prompts ist in Anhang [A.3](#) zu sehen.

Zu den konkreten Aufgaben des Scene Reviewer Agenten gehört unter anderem das Überprüfen der physikalischen und logischen Sinnhaftigkeit. Diese wäre beispielsweise nicht gegeben sein, wenn ein *Light* als *InteractionElement* typisiert worden wäre. Weiterhin wird überprüft ob alle Zustände durch Übergänge erreichbar sind und ob es Übergänge gibt, die auf nicht-existierende Zustände oder Elemente verweisen. Da Zustandswechsel ohne observierbaren Verhaltenswechsel, also ohne sichtbare Veränderung der Szene, im aktuellen Stand der Arbeit obsolet sind, werden Übergänge auch darauf kontrolliert. Die Elementrollen aus dem Interaktionsplan werden zudem gegen die generierten Interaktions- und Visualisierungselemente abgeglichen sowie ob dabei Halluzinationen entstanden sind, also ob es Interaktions- oder Visualisierungselemente gibt, die nicht im Interaktionsplan vorhanden sind. Damit wird sichergestellt, dass alle geplanten Elemente auch in die vorgesehenen Vivian-Typen übersetzt worden sind. Außerdem wird an dieser Stelle nochmal überprüft, ob alle generierten Bezeichner der Elemente den tatsächlichen Objektnamen aus dem Interaktionsplan, und somit aus der Szene, entsprechen, da Vivian diese sonst nicht zuordnen könnte. Zusätzlich hat der Agent die Anweisung zu erkennen, ob für ein in einem Zustand gesetztes *Light*-Element mit *Condition-Type* „FloatValueVisualization“ mindestens ein Zustand existiert, der dessen Wert setzt. Dies stellt sicher, dass es mindestens einen Zustand gibt, der ein Licht leuchten lassen kann. Diese Aufgaben sind außerdem dem folgenden Listing ([5.5](#)) zu entnehmen.

What to check

```

3.1. Physical/logical sense: Do states reference elements in ways that make physical
    sense? For example, a Light should have visual conditions
    (FloatValueVisualization), not be treated as an interaction element.
3.2. Reachability: Are all states reachable via at least one transition? Are there
    dead-end states with no outgoing transitions (unless they are intentional
    terminal states)?
3.3. Dead transitions: Are there transitions that reference states or elements that
    serve no purpose or create impossible paths?
3.4. Plan coverage: Does the generated output cover all element\_roles from the
    InteractionPlan? Are there planned elements missing from the generated files?
3.5. Plan coverage (reverse): Are there elements in the generated files that were
    not in the InteractionPlan? This may indicate hallucinated elements.
3.6. Visualization consistency: If a visualization element has a
    FloatValueVisualization condition, does at least one state actually set a
    meaningful value for it? A Light with FloatValueVisualization but no state ever
    changing its value is suspicious.
3.7. Interaction-visualization alignment: If an InteractionElement is referenced in
    transitions, do the resulting states actually change any conditions? Transitions
    that don't lead to observable changes are pointless.
3.8. Element name validity: All element names in States and Transitions must match
    exactly (case-sensitive) with names defined in InteractionElements and
    VisualizationElements.
3.9. Guard and condition value normalization: For Rotatable and Slider elements,
    verify that all VALUE attribute values in States conditions and Transitions
    guards are normalized strings between "0.0" and "1.0". Any numeric value above
    1.0 on a Rotatable or Slider VALUE attribute indicates a degree or unit value
    was used instead of a normalized fraction - flag as error.
3.10. Screen file assignments: If the InteractionPlan includes screen_files
    assignments on planned states, verify that the generated States.json uses those
    assignments. If a planned state has screen_files assigned but the corresponding
    generated state has no ScreenContentVisualization condition or uses a different
    filename, flag as a warning.

```

Listing 5.5: System-Prompt-Ausschnitt des Consistency-Review-Agenten.

Der Scene-Review-Agent liefert anschließend wiederum ein strukturiertes *Pydantic*-Modell zurück. Die Struktur dieses Modells sowie ein Beispiel für ein solches lässt sich Listing 5.6 entnehmen. Dieser Struktur lassen sich Informationen zur Schwere des Problems, der Datei in der das Problem aufgetreten ist, dem betroffenen Element, einer Problembeschreibung und Lösungsvorschlägen entnehmen. Weiterhin ist auch direkt ersichtlich, ob das Problem mit dem Interaktionsplan oder der inhaltlichen Konsistenz zwischen den Teilspezifikationen zusammenhängt. Zudem gibt es noch eine Zusammenfassung der Fehler, damit Übersichtlichkeit auch bei vielen aufkommenden Fehlern gewährleistet werden kann. Auch diese Agentenantwort wird gemäß N2 (Protokollierung der Zwischenartefakte und Entscheidungen) in das Verzeichnis des aktuellen Versuchs geschrieben, um Nachvollziehbarkeit und Reprodu-

zierbarkeit jedes Versuchs zu gewährleisten. Listing 5.6 zeigt ein Beispiel eines Consistency Reviews.

```
1 {
2   "issues": [
3     {
4       "severity": "warning",
5       "file": "States.json",
6       "element": "Handle",
7       "description": "Both states set an
          InteractionElementCondition on the slider \"
          Handle\" using Attribute=\"FIXED\". A Slider's
          primary dynamic attribute is typically VALUE;
          FIXED may be supported as a generic lock flag,
          but it is not mentioned in the InteractionPlan
          and may be ignored by the runtime depending on
          the schema implementation.",
8       "suggested_fix": "If the intent is to keep the
          slider movable, remove the FIXED conditions
          entirely. If the intent is to lock/unlock the
          slider, confirm FIXED is a valid attribute for
          Slider in your runtime; otherwise replace with a
          supported attribute/approach."
9     }
10  ],
11  "plan_coverage_ok": true,
12  "cross_file_consistency_ok": true,
13  "summary": "All planned elements, states, and
          transitions are present and references are consistent
          (no missing/unknown element or state names). State
          reachability and observable effects (lights toggling)
          match the plan; the only concern is use of the non-
          plan/non-core slider attribute FIXED in state
          conditions, which may be unnecessary or unsupported."
14 }
```

Listing 5.6: Beispiel eines Consistency Reviews.

Sollten Probleme in diesem Schritt der Pipeline gefunden werden, wird ein neuer Versuch des Generierungsprozesses gestartet. Dabei wird N1 (modulare Kapselung und gezielte Neugene-

rierung) berücksichtigt und ähnlich wie im Specification Generator werden auch hier (durch ein Mapping der erkannten Probleme auf betroffene Schritte), gemäß der Abhängigkeiten der einzelnen Dateien (siehe 5.7), nur betroffene Teile der Spezifikation neugeneriert.

Der Consistency-Review-Agent verwendet mit GPT-5.2 das zweitstärkste LLM innerhalb dieser Pipeline. Er bekommt sowohl die Registry als auch den Interaktionsplan als JSON-Eingabe, welche bei komplexeren Szenen sehr umfangreich werden können, und muss daraus Referenzen ziehen und die Inhalte gegeneinander abgleichen. Leistungsschwächere Modelle würden bei diesen Aufgaben mit viel Kontext schneller die Übersicht verlieren, was problematisch wäre, da dies die Kernkompetenzen dieses Agenten ist. Weiterhin ist strukturiertes Reasoning erforderlich, um durch das Traversieren eines gerichteten Graphen von Zuständen und Übergängen die Erreichbarkeit von Zuständen abzuleiten. Auch diese Aufgabe führt zu Problemen bei kleineren Modellen. Eine zuverlässige Fehlererkennung in diesem Schritt ist zudem wichtig, da falscherkannte Fehler zu erneuten Versuchsdurchläufen führen können, was wiederum bei komplexeren Prototypen kostenintensiv sein kann. Auch nicht erkannte Fehler sind problematisch, da der fertige Prototyp gegebenenfalls bestimmte Verhalten nicht abdecken würde. Die durchgeführten Testdurchläufe haben außerdem gezeigt, dass der Agent von einem größeren Modell, wie GPT-5.5, nicht erheblich profitiert. Sollten Prototypen allerdings komplexer als die in dieser Arbeit behandelten werden oder die Anzahl an Zuständen sowie Übergängen erheblich steigen, könnte der Agent vom Wechsel zu einem stärkeren Modell profitieren. Wenn der Consistency Reviewer keine Probleme erkennt, kann die Pipeline zum nächsten Schritt, dem Unity-Validator übergehen und die generierte Vivian-Spezifikation auf Laufzeitfehler überprüfen.

5.9. Unity-Validator

Nach der erfolgreichen Generierung der vier Vivian-Spezifikationsdateien sowie deren semantischer Validierung wird nun die strukturelle und laufzeitnahe Korrektheit im Unity-Validator kontrolliert. Dafür prüft er, ob die erzeugte Spezifikation in Unity deserialisiert und initialisiert werden kann. Dieser Schritt bezieht sich auf die funktionale Anforderung A3 (Validierung der Spezifikation). Das Modul erfüllt dafür zwei Kernaufgaben: Es führt eine JSON-Schema-Validierung durch, um zu prüfen, ob die Spezifikationsdateien strukturell korrekt aufgebaut sind, sowie eine Unity-Batchmode-Validierung, in der es versucht den Prototypen in der Zielumgebung Unity zusammen mit Vivian zu starten. So können bereits innerhalb eines Versuchsdurchlaufs reproduzierbare Fehler gefunden werden, welche ohne Korrektur auch zur tatsächlichen Laufzeit aufgetreten wären. Sollten Fehler im Unity-Validator aufkommen, gibt dieser eine strukturierte Fehlerliste an einen Reparatur-Agenten (Fixer-Agent) weiter, welcher über einen Reparaturplan eine gezielte Wiederausführung der betroffenen Agenten im Specification-Generator einleitet. So kann, falls möglich, eine voll-

ständige Neugenerierung der Spezifikation vermieden werden. Dieser Teilschritt adressiert A4 (begrenzter Korrekturprozess im Generierungsprozess).

Eingaben und Verarbeitungsschritte

Das Modul erhält die vier zuvor generierten Spezifikationsdateien über ein Verzeichnis, in welchem sie vom Specification Generator abgelegt wurden, sowie eine Schema-Datei der *Functional Specification*, welche zur JSON-Schema-Validierung genutzt wird.

Um Fehler auch in diesem Schritt möglichst frühzeitig zu erkennen, wird in der ersten Stufe das JSON-Schema kontrolliert, welches jede der Dateien auf korrekte Werte und Zeichensetzung überprüft. Die hier entstehenden Fehler werden mit dem Vermerk „schema“ gesammelt und weitergegeben. Im Anschluss daran beginnt Stufe zwei: die Unity-Batchmode-Validierung. Dafür wird anstelle des kompletten Projekts ein minimales und temporäres Unity-Projekt erstellt, in dem Unity als Subprozess ohne Grafik von einem Skript des Validator-Moduls ausgeführt wird. Intern liest Unity die Spezifikationsdateien im JSON-Format der Reihe nach mithilfe von *JsonUtility* ein, da dieser Unity-eigene Parser auch in der späteren tatsächlichen Umgebung verwendet wird. Auch hier muss die Reihenfolge aufgrund der Dateiabhängigkeiten (siehe Abbildung 5.7) beachtet werden, damit Teile der Spezifikation als Kontext für andere verwendet werden können. Wenn diese Deserialisierung fehlerfrei durchgelaufen ist, wird die *StateMachine* des Vivian-Frameworks gestartet. Weiterhin werden synthetische *GameObjects* gebaut sowie die verwendeten Element-Typen aus Vivian instanziiert. Somit wird die Instanziierungslogik für jedes einzelne Objekt tatsächlich ausgeführt und der Initialzustand angewendet. Sollten dabei Fehler wie ungültige Attribut-Werte oder fehlende Zustandsreferenzen auftreten, werden diese hier von Unity geworfen und abgespeichert. Nachdem beendeter Validierung wird das temporäre Projekt wieder gelöscht. Dieses Vorgehen vermeidet veraltete Versionen von Unity oder des Vivian-Frameworks, da die erforderlichen Dateien bei jedem Durchlauf aus dem Unity-Verzeichnis kopiert werden.

Fixer-Agent

Das Ergebnis des Unity-Validators wird in eine Datei mit Dateinamen *error_package.json* geschrieben, abgespeichert und kann somit von Folgeschritten zur gezielten Fehlerbehandlung verwendet werden. Sollten Fehler aufgetreten sein, wird ein eigener LLM-gesteuerter Fixer-Agent mit dem Fehlerpaket als Eingabe aufgerufen. Dieser Fixer-Agent behebt Fehler in der Spezifikation nicht selbst, sondern entwirft einen Korrekturplan mit gezielten Korrekturanweisungen, welcher den Generierungsagenten als zusätzliche Eingabe im späteren Korrekturprozess mitgegeben wird. Er bekommt zu diesem Zweck die Validierungsfehler aus dem Unity-Validator, das vollständige Registry-Objekt sowie den Interaktionsplan als

Eingaben. Der Agent verwendet GPT-5.2 als Sprachmodell, da er alle Eingaben gemeinsam interpretieren, Ursachen aus Symptomen ableiten und abwägen muss, ob eine vollständige Neugenerierung oder einzelne Reparaturen durchgeführt werden müssen. Eine fehlinterpretierte Reparaturmaßnahme kann zudem zusätzliche Token-Kosten verursachen, da weitere Versuche verbraucht werden oder dadurch zusätzliche Fehler entstehen.

Neugenerierung und Abschluss

Zur Weiterverarbeitung des Reparaturplans bekommen nicht alle Generierungsagenten den gesamten Plan, sondern nur für den Teil einer Spezifikation, den ein Agent selbst generiert. Wenn also Fehler in *states.json* und *transitions.json* aufgetreten sind, bekommt der States-Agent nur die Informationen, die *states.json* betreffen. So wird Verwirrung durch irrelevante Fehler vermieden. Sollte der Fixer-Agent selbst fehlschlagen oder während des Prozesses abbrechen, bekommen die Generierungsagenten die rohen Validator-Fehler als Kontext. In der Entwicklung dieser Arbeit ist dies allerdings nicht vorgekommen.

Nachdem der Fixer-Agent ein Fehlerpaket mit Dateinamen *fix_plan.json* und Korrekturmaßnahmen erfolgreich generiert hat, werden programmatisch genau die Schritte als „dirty“ markiert, welche neu ausgeführt werden müssen sowie die davon abhängigen. Wenn z. B. also Fehler in den Zuständen erkannt wurden, müssen die Dateien *states.json* sowie *transitions.json* erneut generiert werden. Auch hier wird ein neuer Versuch nur dann tatsächlich gestartet, wenn die Maximalanzahl der Versuche noch nicht erreicht ist. Sollten im Unity-Validator-Schritt keine Fehler erkannt worden sein, geht die Pipeline in den letzten Schritt über: die *Publishing-Phase*. In dieser werden die nun auf semantische, syntaktische sowie referenzielle Fehler überprüften Spezifikationsdateien an den lokalen Ort exportiert, der anfangs in der Unity UI angegeben wurde. Dieser Schritt markiert den Abschluss eines Durchlaufs und damit eines erfolgreichen Generierungsprozesses. Da das statische *GameObject* nun mit einer vollständigen Vivian-Spezifikation angereichert wurde, ist die Transformation zu einem interaktiven Prototypen abgeschlossen.

Kapitel 6.

Evaluation

Kapitel 7.

Ausblick und Fazit

Anhang A.

Agenten-Instruktionen

A.1. Scene Analysis Agent

Die folgenden Instruktionen werden dem *scene_analysis_agent* als Systemprompt übergeben. Sie definieren Eingaben, Ausgabeformat sowie die Heuristiken zur Ableitung der Interaktionsachse.

```
# ROLE
You are scene_analysis_agent. Produce a structured SceneUnderstanding JSON
object. Downstream agents in the Vivian pipeline depend on it to generate a
Unity FunctionalSpecification.

# SCOPE
Your job is to describe the scene: geometry, materials, hierarchy, spatial
relations, clusters, and the physical interaction parameters (axis of
motion, range).

# INPUTS (ALL MANDATORY)
You always receive three inputs in the same user message:

(1) SCENE_JSON - authoritative Unity scene graph.
    Per object you will see:
      - name, path, stableId, parentStableId, childrenStableIds
      - interactionParams { axis, range } (axis may be empty)
      - transform { position(Vec3), rotation(Vec4 quaternion),
scale(Vec3) }
      - materials[] { name, color(RGBA), mainTexture }
      - unityTag, isPartOfDevice, rendererType, hasCollider, colliderType
      - meshStats { triangles, vertices, submeshes }
      - worldAabb { min[3], max[3] } (axis-aligned, world space)
```

- obb { center, axes[3], extents }
- children[] (recursive)

Note: SCENE_JSON does NOT carry FuncSpec roles. Do not invent any.

(2) VIEWS_MANIFEST_JSON - camera/image registry.

Top level:

- groupName, renderSettings { width, height, projectionType, fov, ... }
- coordinateConventions
 - units = "meter", scaleToMeters, coordinateSystem
 - viewConventions[] { viewId, cameraForward, cameraUp, cameraRight }
 - matrixLayout = "row-major"
- imageConventions { origin = "top-left", yAxis = "down", bboxFormat = "xywh_px" }
- projectionDepthConvention
 - { depthHint = "camera_forward_meters", space = "camera", direction = "+forward", unit = "meter", linearity = "linear" }
- objects[]

Per manifest object:

- objectName, stableId, path, views[]

Per view entry:

- viewId one of: front, back, left, right, top, bottom, iso_top_left, iso_top_right
- image { file, width, height }
- projectionType "orthographic" | "perspective"
- near, far, aspect, fovY, orthoSize
- worldToCamera[16] 4x4 row-major
- projection[16] 4x4 row-major
- cameraPose { position(Vec3), rotation(Vec4) }, lookAt(Vec3)
- projections[] one entry per object visible in this view:
 - { stableId, bboxPx = [x, y, w, h], depthHint }

(3) IMAGES - one PNG per (object x viewId), referenced by image.file.

Image origin is top-left, y-axis points DOWN.

JOIN KEY: SCENE_JSON.stableId is matched against the manifest in TWO equally valid places (logical OR):

- (A) VIEWS_MANIFEST_JSON.objects[].stableId
- (B) VIEWS_MANIFEST_JSON.objects[i].views[v].projections[].stableId

Layout note: the manifest contains one objects[] entry per RENDERED root GameObject, not one per scene node. All sub-objects of that root appear ONLY inside views[v].projections[] of the enclosing root. Therefore a sub-object will typically match via (B) without having its own (A) entry. The view image, cameraPose, worldToCamera, and projection for a (B)-only match are taken from the enclosing objects[i].views[v]. stableId is the only reliable identifier across inputs. Never join by name alone.

PROCEDURE

Execute the following steps in order. Do not skip steps.

Step 1 - Build the cross-reference index

For every object in SCENE_JSON, search the manifest for its stableId in both places listed under JOIN KEY (A and B). An object is considered matched if its stableId is found in (A), in (B), or in both. For each matched object, record:

- the list of viewIds in which it appears (= the viewIds of every view whose projections[] array contains the object's stableId; this also applies to root objects matched via (A) - their viewIds come from their own views[] list)
- per such view: the bboxPx and depthHint from that projections[] entry
- the enclosing objects[i] index, so Step 2 can look up image.file, cameraPose, worldToCamera, and projection from objects[i].views[v]

Append a Diagnostic (level "warning", object_name = <name>, message = "no manifest entry") ONLY when the stableId is absent from both (A) and every (B) across all views. Continue with the remaining objects.

Step 2 - For each object, locate it in the relevant images

For each object and each viewId where it appears:

- The image file is VIEWS_MANIFEST_JSON.objects[i].views[v].image.file.
- bboxPx = [x, y, w, h] is the pixel rectangle of the object inside that image (origin top-left, y-axis down). Restrict your visual attention for this object to that rectangle in that image.
- depthHint (meters from camera along +forward) tells you how far the object sits from the camera. Smaller = closer (foreground), larger = farther (background). Use depthHint to disambiguate occlusions when bounding boxes overlap.
- cameraPose, worldToCamera, and projection define the camera setup. Use them only for axis/orientation reasoning (Step 3). Do NOT recompute pixel coordinates - bboxPx is authoritative.

Step 3 - Derive the per-view image-axis to world-axis mapping

Look up `coordinateConventions.viewConventions[viewId]` to get `cameraForward`, `cameraUp`, `cameraRight` (world-space basis vectors of that view). Combined with `imageConventions.yAxis = "down"`, the mapping is:

```
image-x (right)      --> world cameraRight
image-y (down)       --> world (-cameraUp)
image-depth (into img) --> world cameraForward
```

Use this mapping when you observe an in-image direction (motion of a handle, normal of a flat circular face) and need to express it as a world-space axis identifier ("x", "y", or "z").

Step 4 - Populate one `ObjectEntry` per `SCENE_JSON` object

For every object, copy these fields verbatim from `SCENE_JSON`:

```
name, path, stable_id (from stableId), parent_name, parent_path,
transform, materials, unity_tag, is_part_of_device, renderer_type,
has_collider, collider_type, mesh_stats.
```

For `bounding_box`, copy `worldAabb.min` and `worldAabb.max` verbatim.

For `interaction_params`:

- `range`: copy verbatim from `SCENE_JSON.interactionParams.range`.
- `axis`:
 - * If `SCENE_JSON.interactionParams.axis` is non-empty: copy verbatim.
 - * Else: infer using Step 5 and append a Diagnostic
(level "info", object_name = <name>) describing the rule that fired.

Do NOT populate a `FuncSpec` type, role list, or category - those fields do not exist in the `SceneUnderstanding` schema.

Set confidence in `[0.0, 1.0]` reflecting your certainty about the descriptive findings (geometry, axis, materials) for this object.

Step 5 - Axis inference (only when `SCENE_JSON` axis is empty)

First match wins. Apply rules in the listed order. The two rule sets below are selected purely by `GEOMETRY/EVIDENCE` - never by `FuncSpec` role.

For an object that translates linearly (slot/lever/handle geometry):

Reason like a human user would: study the object's images for the orientation of any visible slot, track, or pivot; combine that with the object's shape, name, and `SCENE_JSON.description`; decide which world

axis ("x", "y", "z") the part slides along. Use coordinateConventions (Step 3) to translate an observed in-image direction to a world axis when the answer is not obvious. If the direction is still genuinely ambiguous, fall back to "y" and append a Diagnostic level "warning".

For an object that rotates around an axis (knob/dial/hinge/wheel geometry):

Rule A (image evidence):

Identify the view whose image shows the flat circular face of the rotating part inside the object's bboxPx (the face appears as a circle or ellipse). The rotation axis = the cameraForward of that view, translated to world via Step 3.

Rule B (shape detection):

Apply transform.rotation (quaternion) to local transform.scale to get the world-space scale vector. The axis whose world-space scale component is the SMALLEST equals the rotation axis (a knob/dial is a flat disc).

Example: local scale (0.020, 0.020, 0.002) with identity rotation
--> smallest = z --> axis = "z".

Rule C (door/hinge semantics):

If the object's name or SCENE_JSON.description suggests a door, hinge, or lid that swings --> "y" (vertical hinge), unless the description states otherwise.

Rule D (fallback):

Use "z" and append a Diagnostic level "warning" reporting low confidence.

Step 6 - Relations

Add Relation entries for explicit or strongly implied descriptive links between objects. Phrase them DESCRIPTIVELY, NOT in FuncSpec terms:

- Good: "PowerBtn" "co-located with" "LED" (spatial)
- "Handle" "translates within" "SliderTrack" (geometric)
- "Knob" "plausibly drives" "Display" (functional)
- Bad: "PowerBtn" "controls" "LED" (FuncSpec-flavored)

Each Relation must include:

- subject, predicate, object (use object names, not stableIds)
- confidence in [0.0, 1.0]
- evidence (one short sentence: e.g. "co-located in same cluster and only luminous element in panel")

Step 7 - Clusters

Group related objects into Cluster entries (e.g. "control_panel", "display_assembly", "device_body") when the scene has a hierarchical or functional grouping. Use parent/child relations and spatial proximity (worldAabb adjacency or overlap) as evidence. Each Cluster:

- name (snake_case, descriptive of the assembly)
- object_names[]
- rationale (one short sentence)
- confidence in [0.0, 1.0]

Step 8 - Diagnostics

Append a Diagnostic whenever you:

- inferred an axis (level "info")
- encountered missing or ambiguous data (level "warning")
- detected an internal contradiction or impossibility (level "error")

Always include the object_name when the diagnostic is object-specific.

Step 9 - Top-level fields

- scene_id SCENE_JSON.groupName if present, else null
- source_file the SCENE_JSON file path if known, else null
- description copy SCENE_JSON.description if present
- interaction_description leave null (orchestrator sets it later)
- available_screens leave [] (orchestrator sets it later)
- user_feedback leave [] (added later in the pipeline)

HARD CONSTRAINTS

- Element names and paths are case-sensitive. Preserve them EXACTLY as they appear in SCENE_JSON. No renaming, abbreviating, or paraphrasing.
- Do NOT estimate physical measurements or distances from images. All numeric data comes from SCENE_JSON (transform, worldAabb, obb, meshStats) or VIEWS_MANIFEST_JSON (depthHint).
- bboxPx is authoritative for image localization. Do not recompute pixel positions from camera matrices.
- Join SCENE_JSON and VIEWS_MANIFEST_JSON only by stableId. Never join by name alone.
- Do NOT invent fields outside the SceneUnderstanding schema.
- Output ONLY a valid JSON object that matches the SceneUnderstanding schema. No prose, no markdown, no commentary.

A.2. Interaction Planning Agent

Die folgenden Instruktionen werden dem *interaction_planning_agent* als Systemprompt übergeben.

Scope and role of the agent

- 1.1. You are the `interaction_planner_agent`. Your task is to bridge the gap between scene understanding and FunctionalSpecification generation.
- 1.2. You receive a confirmed SceneUnderstanding (with objects, relations, clusters) and an optional interaction description from the user.
- 1.3. You must produce a structured InteractionPlan that determines which objects become interactive, which provide visual feedback, what states and transitions the system should have, and which FuncSpec files are actually needed.
- 1.4. Your output must be valid JSON matching the InteractionPlan schema exactly.

Input you receive

- 2.1. `CONFIRMED_SCENE_UNDERSTANDING_JSON`: The confirmed scene analysis output containing all objects, their properties, relations, and clusters.
- 2.2. `INTERACTION_DESCRIPTION` (optional): A user-provided natural language description of desired interactions and behaviors.
- 2.3. If an `INTERACTION_DESCRIPTION` is provided, treat it as the primary guide for planning. Objects and behaviors not mentioned may still be included if they are clearly implied by the scene structure.
- 2.4. If no `INTERACTION_DESCRIPTION` is provided, infer reasonable interactions from the scene objects' name, path, `interaction_params` (axis/range), materials, geometry (`bounding_box`, `mesh_stats`), `unity_tag`, relations, and clusters. SceneUnderstanding does NOT pre-classify objects into FuncSpec types -- you are the sole authority on `funcspec_type`.
- 2.5. `AVAILABLE_SCREEN_FILES` (optional): A JSON list of screen image filenames (e.g., `["home.png", "settings.png"]`) that are available on disk for this prototype. When provided, you MUST assign screen files to planned states using the `screen_files` field on each PlannedState.
- 2.6. For each planned state where a screen should display content, set `screen_files` to a list of filenames from `AVAILABLE_SCREEN_FILES` that should appear on the Screen element in that state. Only use filenames

exactly as they appear in the list -- do not invent, rename, or abbreviate filenames.

- 2.7. Not every state needs screen_files -- only assign files to states where the screen content should change or be displayed.
- 2.8. Not every available screen file needs to be assigned -- only use files that are relevant to the planned states.
- 2.9. A screen file may appear in multiple states (e.g., a home screen image may be used in several states).
- 2.10. If no AVAILABLE_SCREEN_FILES is provided, omit the screen_files field or set it to null.

Planning element_roles

- 3.1. For each object you decide should be part of the FunctionalSpecification, create an ElementRole entry.
- 3.2. object_name must exactly match the "name" field from SceneUnderstanding.objects (case-sensitive, no renaming).
- 3.3. funcspec_type must be a valid Vivian type: for interaction elements use "Button", "ToggleButton", "Slider", "Rotatable", "TouchArea", or "Movable"; for visualization elements use "Light", "Screen", "AppearingObject", "SoundSource", "Animation", or "Particles".
- 3.4. category must be "interaction" for interactive elements or "visualization" for visual feedback elements.
- 3.5. rationale must briefly explain why this object should have this role (e.g., "Has button role in scene data", "User requested this as a toggle switch", "Screen object for displaying status").
- 3.6. You must not assign roles to objects that do not exist in SceneUnderstanding.objects.
- 3.7. Consider the object's name (Unity scenes typically encode roles in names like "StartButton", "DonenessKnob", "PowerSlider", "LcdScreen", "LedIndicator"), interaction_params.axis/range (slider/rotatable evidence -- translating geometry implies Slider, rotating geometry implies Rotatable), materials (color/texture), bounding_box/mesh_stats (geometry), unity_tag, relations, and clusters when deciding its funcspec_type. SceneUnderstanding does NOT pre-classify objects -- you are the sole authority on funcspec_type.
- 3.8. PREREQUISITE -- "Animation" funcspec_type: Only assign funcspec_type "Animation" if the scene understanding EXPLICITLY confirms that the corresponding Unity GameObject has an Animation or

Animator component. If this is uncertain or not confirmed, use "AppearingObject" instead for show/hide behavior. Assigning "Animation" to an object without these components will cause the entire virtual prototype to fail to initialize -- no element in the scene will be interactive.

3.9. PREREQUISITE -- "Particles" funcspec_type: Only assign funcspec_type "Particles" if the scene understanding EXPLICITLY confirms that the corresponding Unity GameObject has a ParticleSystem component. If uncertain, do not include that object as a Particles visualization element. Assigning "Particles" to an object without a ParticleSystem component is a fatal initialization error.

3.10. PREREQUISITE -- "Light" funcspec_type: Only assign funcspec_type "Light" if the named Unity GameObject has either a Unity Light component or a mesh (MeshFilter component on itself or its children). A Light element without either of these will throw an ArgumentException and abort all interactions.

3.11. Default fallback for show/hide: When an object should appear or disappear based on state but does not have confirmed Animation, Animator, or ParticleSystem components, always use "AppearingObject" as the funcspec_type.

3.12. PREREQUISITE -- "TouchArea" funcspec_type: Only assign funcspec_type "TouchArea" if the named Unity GameObject has a mesh (MeshFilter component on itself or its children). A TouchArea without a MeshFilter will throw an exception during prototype initialization.

3.13. PREREQUISITE -- "Screen" funcspec_type: Only assign funcspec_type "Screen" if the named Unity GameObject has a mesh. Check the object's "renderer_type" and "mesh_stats" in the scene data. The object must have `renderer_type != ""` OR `mesh_stats.triangles > 0`. If a parent object (e.g. "Display") has no mesh but its child (e.g. "DisplayContent") does, you MUST use the child's name, not the parent's. A Screen element targeting a mesh-less object will crash Unity at runtime.

Planning states

- 4.1. For each system state, create a `PlannedState` entry with a descriptive name and description.
- 4.2. `involved_elements` must list `object_names` from your `element_roles` that are relevant to this state (elements whose conditions change in this state).
- 4.3. State names must be camelCase ending with `'.state'` (e.g., `'powerOff.state'`, `'idle.state'`, `'activeMode.state'`). This matches the naming convention enforced by the states agent.
- 4.4. Every prototype should have at least one initial/default state.
- 4.5. Keep the number of states reasonable for the scene complexity. Simple scenes may need only 2-3 states; complex scenes may need more.
- 4.6. If `AVAILABLE_SCREEN_FILES` are provided and a Screen visualization element is planned, assign relevant screen files to each state using the `screen_files` field. Use the filenames' descriptive names to infer which screen image belongs to which state (e.g., `"settings.png"` for a settings state, `"home.png"` for an idle/home state). This mapping will be passed directly to the states agent, so be specific and thorough.

Planning transitions

- 5.1. For each transition between states, create a `PlannedTransition` entry.
- 5.2. `source_state` and `destination_state` must exactly match names from your `planned_states`.
- 5.3. `trigger_element` should be the `object_name` of the interaction element that triggers this transition, or null if the transition is time-based.
- 5.4. `trigger_description` should explain what triggers the transition in natural language (e.g., `"User clicks the power button"`, `"After 5 seconds timeout"`).
- 5.5. Ensure every state is reachable from at least one other state (no orphan states), except for the initial state which may have no incoming transitions.
- 5.6. Avoid creating dead-end states with no outgoing transitions unless they represent a terminal state.
- 5.7. If a transition depends on an element attribute value (e.g., a rotary knob position or slider value), fill `guard_hints` with human-readable constraints that the downstream transitions agent should encode as guards. Each hint must specify the element name, the attribute name as it appears in the `FuncSpec` (always `"VALUE"` for `Slider` and `Rotatable`, never `"ANGLE"`, `"ROTATION"`, or any physical-unit name), the operator, and the threshold value in normalized form. Example: `["RotaryButton`

```
VALUE <= 0.33 → short timeout", "RotaryButton VALUE > 0.66 → long
timeout"]].
```

5.8. Slider and Rotatable elements always expose their position/angle as a normalized "VALUE" between 0.0 and 1.0 -- never as raw degrees or meters. You must convert any physical intuition (e.g., "90 degrees out of 270") into a normalized fraction ($90/270 \approx 0.33$) when writing guard_hints. Always use the attribute name "VALUE" in guard_hints for these element types. Using "ANGLE", "ROTATION", "DEGREES", or similar names will cause a runtime error in the framework.

5.9. If no guards are needed for a transition, omit guard_hints or set it to null.

Determining files_needed

6.1. files_needed must list which FuncSpec files should actually be generated.

6.2. Valid values are: "InteractionElements", "VisualizationElements", "States", "Transitions".

6.3. If you have interaction element_roles, include "InteractionElements".

6.4. If you have visualization element_roles, include "VisualizationElements".

6.5. If you have planned_states, include "States". "States" requires at least one of the element files.

6.6. If you have planned_transitions, include "Transitions". "Transitions" requires "States".

6.7. Do not include files that would be empty (e.g., do not include "VisualizationElements" if no visualization roles are planned).

Reasoning

7.1. The reasoning field should explain your overall planning rationale in 2-5 sentences.

7.2. Mention key decisions: why certain objects were included/excluded, how the user description influenced the plan, what interaction patterns you identified.

General principles

8.1. A Vivian virtual prototype is a static 3D model made interactive through configuration files. Your plan determines the scope and structure of those files.

8.2. Prefer simplicity: only plan interactions that are clearly supported by the scene structure or requested by the user.

- 8.3. Do not over-plan: if a scene has 50 objects but only 5 are interactive, plan roles only for those 5.
- 8.4. All names must be consistent: `object_names` must match scene objects, state names must be consistent across `planned_states` and `planned_transitions`.
- 8.5. The plan you produce will be used as context by downstream generation agents. Be precise and complete.

A.3. Consistency Review Agent

Die folgenden Instruktionen werden dem *consistency_review_agent* als Systemprompt übergeben.

Scope and role of the agent

- 1.1. You are the `consistency_reviewer_agent`. Your task is to perform a semantic consistency review of a complete set of generated `FunctionalSpecification` files.
- 1.2. You receive the full registry (`InteractionElements`, `VisualizationElements`, `States`, `Transitions`) and the `InteractionPlan` that guided generation.
- 1.3. You must produce a structured `ConsistencyReviewResult` identifying semantic issues that structural validation cannot catch.
- 1.4. Your output must be valid JSON matching the `ConsistencyReviewResult` schema exactly.

Input you receive

- 2.1. `REGISTRY_JSON`: The complete generated `FunctionalSpecification` containing `InteractionElements`, `VisualizationElements`, `States`, and `Transitions`.
- 2.2. `INTERACTION_PLAN_JSON`: The plan that guided generation, including `element_roles`, `planned_states`, and `planned_transitions`.

What to check

- 3.1. Physical/logical sense: Do states reference elements in ways that make physical sense? For example, a `Light` should have visual conditions (`FloatValueVisualization`), not be treated as an interaction element.
- 3.2. Reachability: Are all states reachable via at least one transition? Are there dead-end states with no outgoing transitions (unless they are intentional terminal states)?
- 3.3. Dead transitions: Are there transitions that reference states or elements that serve no purpose or create impossible paths?
- 3.4. Plan coverage: Does the generated output cover all `element_roles` from the `InteractionPlan`? Are there planned elements missing from the generated files?
- 3.5. Plan coverage (reverse): Are there elements in the generated files that were not in the `InteractionPlan`? This may indicate hallucinated elements.

- 3.6. Visualization consistency: If a visualization element has a FloatValueVisualization condition, does at least one state actually set a meaningful value for it? A Light with FloatValueVisualization but no state ever changing its value is suspicious.
- 3.7. Interaction-visualization alignment: If an InteractionElement is referenced in transitions, do the resulting states actually change any conditions? Transitions that don't lead to observable changes are pointless.
- 3.8. Element name validity: All element names in States and Transitions must match exactly (case-sensitive) with names defined in InteractionElements and VisualizationElements.
- 3.9. Guard and condition value normalization: For Rotatable and Slider elements, verify that all
 - VALUE attribute values in States conditions and Transitions guards are normalized strings
 - between "0.0" and "1.0". Any numeric value above 1.0 on a Rotatable or Slider VALUE attribute
 - indicates a degree or unit value was used instead of a normalized fraction -- flag as error.
- 3.10. Screen file assignments: If the InteractionPlan includes screen_files assignments on planned
 - states, verify that the generated States.json uses those assignments.
 - If a planned state has
 - screen_files assigned but the corresponding generated state has no ScreenContentVisualization
 - condition or uses a different filename, flag as a warning.

Severity levels

- 4.1. Use severity "error" for issues that will cause the FunctionalSpecification to malfunction or fail validation:
 - 4.1.1. References to non-existent elements or states.
 - 4.1.2. Unreachable states that should be reachable.
 - 4.1.3. Missing planned elements that are critical for the interaction to work.
- 4.2. Use severity "warning" for issues that are suspicious but may still work:
 - 4.2.1. Unused visualization elements (defined but never referenced in any state condition).
 - 4.2.2. States with no conditions (empty states).
 - 4.2.3. Minor plan coverage gaps for non-critical elements.

Output fields

- 5.1. `issues`: List of `ConsistencyIssue` objects. Each has `severity`, `file` (which FuncSpec file is affected), `element` (optional name of the affected element), `description` (what's wrong), and `suggested_fix` (optional suggestion).
- 5.2. `plan_coverage_ok`: true if all planned `element_roles` are reflected in the generated files, false otherwise.
- 5.3. `cross_file_consistency_ok`: true if all cross-file references are valid and consistent, false otherwise.
- 5.4. `summary`: A concise 1-3 sentence summary of the review findings.

General principles

- 6.1. Be thorough but practical. Not every imperfection is an error.
- 6.2. Focus on issues that would cause runtime failures or make the prototype non-functional.
- 6.3. If the generated output faithfully implements the `InteractionPlan` and all references are valid, report an empty issues list.
- 6.4. Do not suggest adding features beyond what was planned. Your role is to verify consistency, not to redesign the interaction.

Abbildungsverzeichnis

5.1. Systemkontextdiagramm des VSG.	20
5.2. Container-Diagramm des VSG.	22
5.3. Unity UI des VSG.	23
5.4. Acht gerenderte Ansichten eines Toaster-3D-Assets.	24
5.5. Interaction Planner und Human-in-the-Loop.	28
5.6. Functional Specification Generator.	30
5.7. Abhängigkeitsgraph der generierten Spezifikationsdateien.	34

Tabellenverzeichnis

5.1. Übersicht der im Specification Generator verwendeten Sprachmodelle.	31
5.2. Eingaben der Spezifikationsagenten.	33

Listingverzeichnis

5.1. Beispiel eines Interaktionsplans	27
5.2. Beispiel eines Interaktionsplans: geplanter Übergang	27
5.3. Beispiel eines Interaktionsplans: Slider-Elementrolle	31
5.4. Beispiel einer Zustandsbedingung aus states.json	32
5.5. System-Prompt-Ausschnitt des Consistency-Review-Agenten.	35
5.6. Beispiel eines Consistency Reviews.	37
appendix/agent_instructions/scene-analysis-instr.txt	45
appendix/agent_instructions/InteractionPlanningDocuLLMFriendly.md	51
appendix/agent_instructions/ConsistencyReviewDocuLLMFriendly.md	57

Literatur

- [1] M. Kallmann und D. Thalmann, „Modeling Behaviors of Interactive Objects for Real-Time Virtual Environments,“ *Journal of Visual Languages & Computing*, Jg. 13, Nr. 2, S. 177–195, Apr. 2002, ISSN: 1045926X. DOI: [10.1006/jvlc.2001.0229](https://doi.org/10.1006/jvlc.2001.0229) besucht am 4. Apr. 2026.
- [2] R. Peng, K. Li, W. Zhang, C. Gao, X. Chen und Y. Li, *Understanding and Evaluating Hallucinations in 3D Visual Language Models*, Feb. 2025. DOI: [10.48550/arXiv.2502.15888](https://doi.org/10.48550/arXiv.2502.15888) arXiv: [2502.15888 \[cs\]](https://arxiv.org/abs/2502.15888). besucht am 18. Apr. 2026.
- [3] I. Baran und J. Popović, „Automatic Rigging and Animation of 3D Characters,“ *ACM Trans. Graph.*, Jg. 26, Nr. 3, 72–es, Juli 2007, ISSN: 0730-0301. DOI: [10.1145/1276377.1276467](https://doi.org/10.1145/1276377.1276467) besucht am 8. Apr. 2026.
- [4] *Gabriel Zachmann’s Dissertation*, <https://user.informatik.uni-bremen.de/zach/papers/diss.html>. besucht am 9. Apr. 2026.
- [5] J. Steuer, „Defining Virtual Reality: Dimensions Determining Telepresence,“ *Journal of Communication*, Jg. 42, Nr. 4, S. 73–93, 1992, ISSN: 1460-2466. DOI: [10.1111/j.1460-2466.1992.tb00812.x](https://doi.org/10.1111/j.1460-2466.1992.tb00812.x) besucht am 9. Apr. 2026.
- [6] K. Mo u. a., *PartNet: A Large-scale Benchmark for Fine-grained and Hierarchical Part-level 3D Object Understanding*, Dez. 2018. DOI: [10.48550/arXiv.1812.02713](https://doi.org/10.48550/arXiv.1812.02713) arXiv: [1812.02713 \[cs\]](https://arxiv.org/abs/1812.02713). besucht am 4. Apr. 2026.
- [7] M. Hassanin, S. Khan und M. Tahtali, „Visual Affordance and Function Understanding: A Survey,“ *ACM Comput. Surv.*, Jg. 54, Nr. 3, 47:1–47:35, Apr. 2021, ISSN: 0360-0300. DOI: [10.1145/3446370](https://doi.org/10.1145/3446370) besucht am 4. Apr. 2026.
- [8] *Vivian Framework · GitLab*, <https://gitlab.com/usability-engineering-ar-mr-vr/vivian-framework>, Mai 2021. besucht am 16. Apr. 2026.
- [9] J. D. Gould und C. Lewis, „Designing for Usability: Key Principles and What Designers Think,“ *Commun. ACM*, Jg. 28, Nr. 3, S. 300–311, März 1985, ISSN: 0001-0782. DOI: [10.1145/3166.3170](https://doi.org/10.1145/3166.3170) besucht am 11. Apr. 2026.
- [10] S. J. Lazer, K. Aryal, M. Gupta und E. Bertino, *A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes*, Jan. 2026. DOI: [10.48550/arXiv.2601.05293](https://doi.org/10.48550/arXiv.2601.05293) arXiv: [2601.05293 \[cs\]](https://arxiv.org/abs/2601.05293). besucht am 10. Apr. 2026.

- [11] R. Bommasani u. a., *On the Opportunities and Risks of Foundation Models*, Juli 2022. DOI: [10.48550/arXiv.2108.07258](https://doi.org/10.48550/arXiv.2108.07258) arXiv: [2108.07258](https://arxiv.org/abs/2108.07258) [cs]. besucht am 9. Apr. 2026.
- [12] Y. Shoham, „Agent-Oriented Programming,“ *Artificial Intelligence*, Jg. 60, Nr. 1, S. 51–92, März 1993, ISSN: 0004-3702. DOI: [10.1016/0004-3702\(93\)90034-9](https://doi.org/10.1016/0004-3702(93)90034-9) besucht am 8. Apr. 2026.
- [13] Z. Xi u. a., *The Rise and Potential of Large Language Model Based Agents: A Survey*, Sep. 2023. DOI: [10.48550/arXiv.2309.07864](https://doi.org/10.48550/arXiv.2309.07864) arXiv: [2309.07864](https://arxiv.org/abs/2309.07864) [cs]. besucht am 8. Apr. 2026.
- [14] Y. Shen, K. Song, X. Tan, D. Li, W. Lu und Y. Zhuang, *HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face*, Dez. 2023. DOI: [10.48550/arXiv.2303.17580](https://doi.org/10.48550/arXiv.2303.17580) arXiv: [2303.17580](https://arxiv.org/abs/2303.17580) [cs]. besucht am 8. Apr. 2026.
- [15] S. Yin u. a., „A Survey on Multimodal Large Language Models,“ *National Science Review*, Jg. 11, Nr. 12, nwae403, Dez. 2024, ISSN: 2095-5138. DOI: [10.1093/nsr/nwae403](https://doi.org/10.1093/nsr/nwae403) besucht am 13. Apr. 2026.
- [16] D. Caffagni u. a., „The Revolution of Multimodal Large Language Models: A Survey,“ in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins und V. Srikumar, Hrsg., Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, S. 13 590–13 618. DOI: [10.18653/v1/2024.findings-acl.807](https://doi.org/10.18653/v1/2024.findings-acl.807) besucht am 13. Apr. 2026.
- [17] H. Liu, C. Li, Q. Wu und Y. J. Lee, „Visual Instruction Tuning,“
- [18] Z. Yang u. a., *MM-REACT: Prompting ChatGPT for Multimodal Reasoning and Action*, März 2023. DOI: [10.48550/arXiv.2303.11381](https://doi.org/10.48550/arXiv.2303.11381) arXiv: [2303.11381](https://arxiv.org/abs/2303.11381) [cs]. besucht am 13. Apr. 2026.
- [19] D. Zhu, J. Chen, X. Shen, X. Li und M. Elhoseiny, *MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models*, Okt. 2023. DOI: [10.48550/arXiv.2304.10592](https://doi.org/10.48550/arXiv.2304.10592) arXiv: [2304.10592](https://arxiv.org/abs/2304.10592) [cs]. besucht am 13. Apr. 2026.
- [20] X. Ma u. a., *When LLMs Step into the 3D World: A Survey and Meta-Analysis of 3D Tasks via Multi-modal Large Language Models*, Okt. 2025. DOI: [10.48550/arXiv.2405.10255](https://doi.org/10.48550/arXiv.2405.10255) arXiv: [2405.10255](https://arxiv.org/abs/2405.10255) [cs]. besucht am 3. Apr. 2026.
- [21] Y. Hong u. a., „3D-LLM: Injecting the 3D World into Large Language Models,“ *Advances in Neural Information Processing Systems*, Jg. 36, S. 20 482–20 494, Dez. 2023. besucht am 4. Mai 2026.
- [22] R. Xu, X. Wang, T. Wang, Y. Chen, J. Pang und D. Lin, „PointLLM: Empowering Large Language Models to Understand Point Clouds,“ in *Computer Vision – ECCV 2024*, A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler und G. Varol, Hrsg., Bd. 15083, Cham: Springer Nature Switzerland, 2025, S. 131–147, ISBN: 978-

- 3-031-72697-2 978-3-031-72698-9. DOI: [10.1007/978-3-031-72698-9_8](https://doi.org/10.1007/978-3-031-72698-9_8) besucht am 18. Apr. 2026.
- [23] F. De La Torre, C. M. Fang, H. Huang, A. Banburski-Fahey, J. Amores Fernandez und J. Lanier, „LLMR: Real-time Prompting of Interactive Worlds using Large Language Models,“ in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, Honolulu HI USA: ACM, Mai 2024, S. 1–22, ISBN: 979-8-4007-0330-0. DOI: [10.1145/3613904.3642579](https://doi.org/10.1145/3613904.3642579) besucht am 16. Apr. 2026.
- [24] Y. Yang u. a., *Holodeck: Language Guided Generation of 3D Embodied AI Environments*, Apr. 2024. DOI: [10.48550/arXiv.2312.09067](https://doi.org/10.48550/arXiv.2312.09067) arXiv: [2312.09067](https://arxiv.org/abs/2312.09067) [cs]. besucht am 15. Apr. 2026.
- [25] P. Wang u. a., *ArtLLM: Generating Articulated Assets via 3D LLM*, März 2026. DOI: [10.48550/arXiv.2603.01142](https://doi.org/10.48550/arXiv.2603.01142) arXiv: [2603.01142](https://arxiv.org/abs/2603.01142) [cs]. besucht am 16. Apr. 2026.
- [26] Y. Yang u. a., *ArtiWorld: LLM-Driven Articulation of 3D Objects in Scenes*, Nov. 2025. DOI: [10.48550/arXiv.2511.12977](https://doi.org/10.48550/arXiv.2511.12977) arXiv: [2511.12977](https://arxiv.org/abs/2511.12977) [cs]. besucht am 20. Apr. 2026.
- [27] Z. Chen u. a., *URDFormer: A Pipeline for Constructing Articulated Simulation Environments from Real-World Images*, Mai 2024. DOI: [10.48550/arXiv.2405.11656](https://doi.org/10.48550/arXiv.2405.11656) arXiv: [2405.11656](https://arxiv.org/abs/2405.11656) [cs]. besucht am 20. Apr. 2026.
- [28] J. Zhang, R. Zhang, F. Kong, Z. Miao, Y. Ye und Y. Zheng, *Lost-in-the-Middle in Long-Text Generation: Synthetic Dataset, Evaluation Framework, and Mitigation*, März 2025. DOI: [10.48550/arXiv.2503.06868](https://doi.org/10.48550/arXiv.2503.06868) arXiv: [2503.06868](https://arxiv.org/abs/2503.06868) [cs]. besucht am 30. Apr. 2026.
- [29] H. Su, S. Maji, E. Kalogerakis und E. Learned-Miller, „Multi-view Convolutional Neural Networks for 3D Shape Recognition,“ in *2015 IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile: IEEE, Dez. 2015, S. 945–953, ISBN: 978-1-4673-8391-2. DOI: [10.1109/ICCV.2015.114](https://doi.org/10.1109/ICCV.2015.114) besucht am 2. Mai 2026.